

Prof. Dr.-Ing. A. Siggelkow

Specification - **RWU-RV64I**



Hochschule Ravensburg-Weingarten
University of Applied Sciences

B

Chapter 1

Requirements

Table 1.1: Functional Requirements

Requirement	ID	Importance	Verifiable	Description	Remarks
General					
Gen.: #persons	G01	High	VHDL-TB	The number of persons in a room must be counted.	
Gen.: max	G02	High	VHDL-TB	The number of persons in a room must not exceed a given limit.	
Gen.: only one pers.	G03	High	?	Only one person can either enter or leave the room at a time.	Check test
Gen.: three light sensors	G04	Medium	VHDL-TB	Along the doorway, there are three light-curtains to allow direction-tracking of possible visitors.	Why not only two?

Chapter 2

Document Overview

2.1 Validity of this document

This document is valid for **RWU-RV64I** version:

- V1.0: First samples
- V1.x: Revision

2.2 Target Specification Status

This **RWU-RV64I** target specification is initially derived from the **RV64I** specification. So readers being familiar with the **RV64I** architecture concepts and hardware building blocks will find most of them unchanged within this document.

This **RWU-RV64I** document represents and describes all features of the **RWU-RV64I** and is adequate as a standalone reference.

This release of the specification has the status of an target specification.

2.3 Major Changes since last Revision

Table 2.1: Spec Changes as compared to the previous **RWU-RV64I** Spec Revision

History		
Target Spec.	Current version : 1.0, 2022-11-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2022-11-22		
	2022-11-22	First draft (AS)

Contents

1	Requirements	1
2	Document Overview	3
2.1	Validity of this document	3
2.2	Target Specification Status	3
2.3	Major Changes since last Revision	4
2.4	Glossary	13
2.5	Naming Conventions	13
2.6	Document List	14
3	Product Overview	15
3.1	Top Level View	15
3.1.1	Introduction	15
3.1.2	Key Features	16
3.1.3	Functional Block Diagram	16
3.1.4	Package	17
3.2	System View	17
3.3	Architecture Concepts Overview	18
3.3.1	Technology	18
3.3.2	System Memory Concept	18
3.3.3	Software Architecture	18
3.3.4	Interprocessor Communication Concept	18
3.3.5	Bus Concept	19
3.3.6	Clock Concept	19
3.3.7	System Control Concept	19
3.3.8	Interrupt Concept	19
3.3.9	Debug Concept	19
3.3.10	Power Management	19
3.3.11	Protection and Security Concept	19
3.4	Functional Block Overview	20
3.4.1	RISC-V Core	20

3.4.2	Instruction Memory	20
3.4.3	Data Memory	20
3.4.4	Wishbone Peripherals	20
3.4.5	Debug	20
4	Architecture Concepts	21
4.1	Technology	22
4.2	CPU Concept	22
4.3	Bus Concept	26
4.3.1	Overview	26
4.3.2	Wishbone B4 Classic Protocol	27
4.3.3	Wishbone Slave BPI (as_slave_bpi)	30
4.3.4	Wishbone Master BPI (as_master_bpi)	35
4.3.5	Bus Interconnect	39
4.3.6	Comparison: Slave BPI vs. Master BPI	40
4.4	Interrupt Concept	41
4.5	Clock Concept	41
4.6	System Control Concept	41
4.7	Power Concept	42
4.8	Memory Concept	42
4.9	System Protection and Security Concept	42
4.10	Debug Concept	43
4.11	Register Concept	43
5	RWU-RV64I Core	45
5.1	ALU	45
5.1.1	History of ALU	48
5.1.2	Functional Overview	48
5.1.3	Structural Overview	49
5.1.4	Service Requests of the ALU	55
5.1.5	The ALU Peripheral Kernel	55
5.1.6	Registers	57
5.2	Instruction Decoder	58
5.2.1	History of Instruction Decoder	61
5.2.2	Functional Overview	61
5.2.3	Structural Overview	62
5.2.4	Service Requests of the Instruction Decoder	73
5.2.5	The InstrDec Peripheral Kernel	74
5.2.6	Registers	76

6	Peripherals	77
6.1	GPIO	77
6.1.1	System Integration of GPIO0	77
6.1.2	History of GPIO	81
6.1.3	Functional Overview	81
6.1.4	Structural Overview	82
6.1.5	Service Requests of the GPIO	89
6.1.6	The GPIO Peripheral Kernel	89
6.1.7	Registers	90
6.2	Instruction Memory (I-Mem)	92
6.2.1	System Integration of I-Mem0	92
6.2.2	History of I-Mem	95
6.2.3	Functional Overview	95
6.2.4	Structural Overview	96
6.2.5	Service Requests of the I-Mem	96
6.2.6	The I-Mem Peripheral Kernel	96
6.2.7	Registers	96
6.3	QSPI	97
6.3.1	System Integration of QSPI0	97
6.3.2	History of QSPI	101
6.3.3	Functional Overview	101
6.3.4	Structural Overview	104
6.3.5	Service Requests of the QSPI	110
6.3.6	The QSPI Peripheral Kernel	110
6.3.7	Registers	114
7	Test and Debug	131
8	Electrical Characteristics	133
9	Memory Map and Registers	135

List of Figures

3.1	RWU-RV64I Overview	16
3.2	Simplified Block diagram	17
4.1	Enable with a Pulse	21
4.2	Wishbone Classic Single Transfers: Write – Read – Write	29
4.3	as_slave_bpi – Internal Signal Flow	33
4.4	as_master_bpi – Internal Signal Flow	38
6.1	GPIO asSlaveBPI Peripheral	83
6.2	GPIO Kernel	90
6.3	QSPI asSlaveBPI Peripheral	104
6.4	QSPI Kernel	114
6.5	Interrupt Request Flow	125

List of Tables

1.1	Functional Requirements	2
2.1	Spec Changes as compared to the previous RWU-RV64I Spec Revision	4
2.2	Glossary Type Descriptions	13
2.3	Alias Names for Cores and other Objects in the Spec	14
2.4	History	14
2.5	Document List	14
3.1	History	15
3.2	History	17
3.3	History	18
3.4	History	20
4.1	History	22
4.2	History	22
4.3	Notations for Instructions	23
4.4	RV32I Base Integer Instruction Set [2], [3], [1]	23
4.5	RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]	25
4.6	History	26
4.7	Wishbone Bus Parameters	27
4.8	Wishbone Signal Naming	28
4.9	History of as_slave_bpi	30
4.10	Functional Configuration of as_slave_bpi	30
4.11	I/O of as_slave_bpi	31
4.12	as_slave_bpi Internal Signals	33
4.13	History of as_master_bpi	35
4.14	Functional Configuration of as_master_bpi	35
4.15	I/O of as_master_bpi	36
4.16	as_master_bpi Internal Signals	38

4.17	Slave BPI vs. Master BPI – Key Differences	40
4.18	History	41
4.19	History	41
4.20	History	41
4.21	History	42
4.22	History	42
4.23	History	42
4.24	History	43
4.25	History	43
5.1	Authors	45
5.2	History	46
5.3	ALU0 Feature Set	46
5.4	HW Interfaces of Module ALU0	47
5.5	History	48
5.6	ALU0 I/O	50
5.7	RV32I Base Integer Instruction Set [2], [3], [1]	55
5.8	RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]	57
5.9	Authors	58
5.10	History	58
5.11	InstrDec0 Feature Set	59
5.12	HW Interfaces of Module InstrDec0	60
5.13	History	61
5.14	InstrDec0 I/O	63
5.15	RV32I/RV64I Base Integer Instruction Set [2], [3], [1]	75
6.1	Authors	77
6.2	History	78
6.3	GPIO0 Feature Set	78
6.4	HW Interfaces of Module GPIO0	79
6.5	Register List of Module GPIO0	80
6.6	History	81
6.7	GPIO Kernel I/O	85
6.8	GPIO BPI I/O	86
6.9	GPIO BPI Internals	87
6.10	GPIO Top I/O	87
6.11	GPIO Top Signals	88
6.12	GPIO Service Requests	89
6.13	GPIO Register List	91
6.14	Authors	92

6.15	History	93
6.16	I-Mem0 Feature Set	93
6.17	HW Interfaces of Module I-Mem0	94
6.18	Register List of Module I-Mem0	94
6.19	History	95
6.20	I-Mem Kernel I/O	96
6.21	Authors	97
6.22	History	97
6.23	QSPI0 Feature Set	98
6.24	HW Interfaces of Module QSPI0	99
6.25	Register List of Module QSPI0	100
6.26	History	101
6.27	Reference NOR-Flash Devices	103
6.28	QSPI Kernel I/O (1)	105
6.29	QSPI Kernel I/O (2)	106
6.30	QSPI Kernel I/O (3)	107
6.31	QSPI BPI I/O	108
6.32	QSPI BPI Internals	109
6.33	QSPI Top I/O	109
6.34	QSPI Top Signals	110
6.35	QSPI Service Requests (updated)	110
6.36	QSPI Register List	115
6.37	CLKDIV Examples ($f_{kernel} = 125$ MHz)	123

2.4 Glossary

Table 2.2: Glossary Type Descriptions

Type	Description
UART	Universal Asynchronous Receiver Transmitter
Bus	Binary Unit System
RGB	Red-Green-Blue. A color mixing system.
VGA	Video Graphics Array. A computer graphics standard.

2.5 Naming Conventions

Table 2.3: Alias Names for Cores and other Objects in the Spec

Object	Names also used in the spec
ARM926EJ-S TM	ARM, Controller, Micro controller unit, MCU, Processor, CPU
Bus	Binary Unit System

2.6 Document List

Table 2.4: History

History		
Target Spec.	Current version : 1.0, 2022-11-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2022-11-22		
	2022-11-22	First draft (AS)

The following documents are referred to in this document. They are either available in a separate attachment directory or are general standards.

Table 2.5: Document List

Nr	Doc Type	Version	Date	Filename	Attached
1	ARM engineering specification	A09	15.10.2024	ARM926EJ-S_EngSpec_A09	No
2	ARM926 technical reference manual	-	16.10.2024	DDI0198B_926	Yes

Chapter 3

Product Overview

3.1 Top Level View

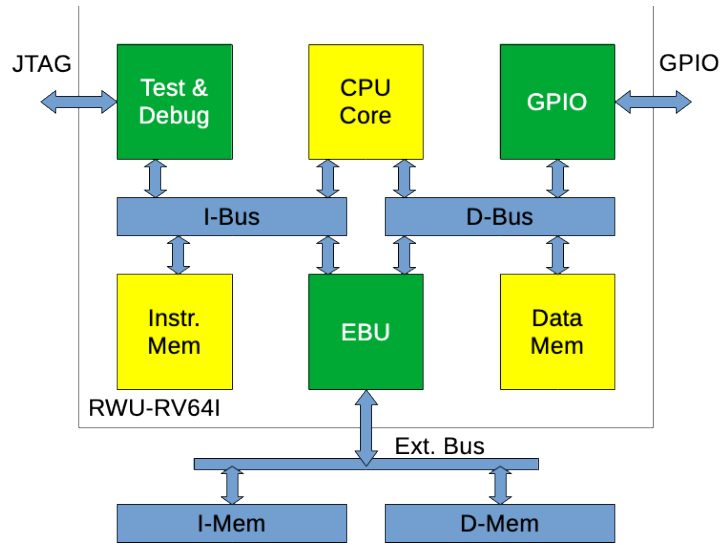
Table 3.1: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.1.1 Introduction

The **RWU-RV64I** is a RISC-V processor with the integer 64 bit instruction set.

Figure 3.1 shows the top level of the chip.

Figure 3.1: **RWU-RV64I** Overview

3.1.2 Key Features

First version, single cycle, on-chip memory only:

- RISC-V RV64I core
- Instruction memory: 65 kB
- Data memory: 65 kB
- JTAG, for loading the I-Mem

3.1.3 Functional Block Diagram

This block diagram shows only the most important architectural features. To maintain legibility some aspects have been simplified. More details of architecture, concepts and block integration are contained in other chapters of this specification.

Figure 3.2 shows a simplified block diagram of the chip.

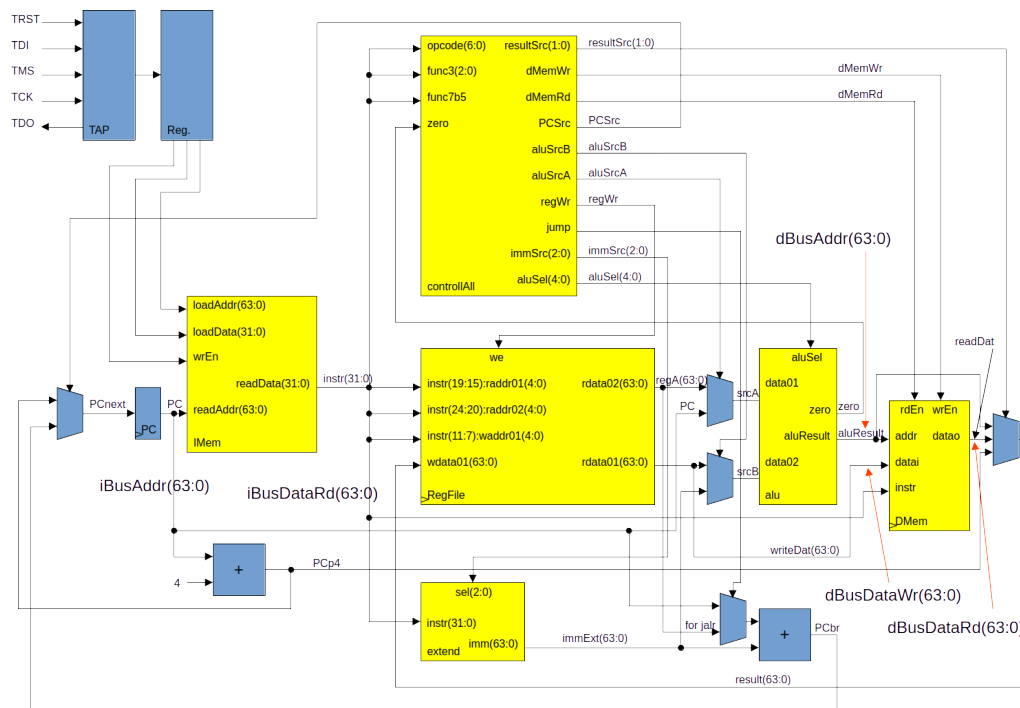


Figure 3.2: Simplified Block diagram

3.1.4 Package

The package is not defined yet.

3.2 System View

Table 3.2: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

The system is not defined yet.

3.3 Architecture Concepts Overview

Table 3.3: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.3.1 Technology

Europractice 90 nm technology.

Infineon's 90 nm (70 nm physical gate length) CMOS technology is designed for digital and analog circuit design. It offers advanced possibilities for low power design (triple well, low leakage transistor types). The technology has 4-9 layers of copper metallization. It offers an increase in peak performance of more than 50% and a reduction of active power consumption of more than a factor of 2. Compared to the predecessor technology twice the cell density for standard cells can be achieved in L90.

The nominal core voltage is 1.2 V, IO devices have a nominal supply voltage of 2.5 V or 1.8 V.

3.3.2 System Memory Concept

The first version has an on-chip instruction memory of ?? MB and an on-chip data memory of ?? MB. The technology is ??.

3.3.3 Software Architecture

The RISC-V compiler ??? is supported.

3.3.4 Interprocessor Communication Concept

Planned: UART and (Q)SPI.

3.3.5 Bus Concept

The main bus system is compliant to the Wishbone specification.

There is on WB bus for the instructions and one for the data.

3.3.6 Clock Concept

Not defined yet.

Until now, there is only one clock input. For a next generation, with external NOR flash memory for instruction and NAND flash for data, a QSPI interface will be needed. This requires a faster clock than the CPU clock.

3.3.7 System Control Concept

System control will be needed for:

- Reset and boot
- Interrupt

3.3.8 Interrupt Concept

Must be implemented.

3.3.9 Debug Concept

Must be implemented.

3.3.10 Power Management

Must be implemented.

3.3.11 Protection and Security Concept

Must be implemented.

3.4 Functional Block Overview

Table 3.4: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.4.1 RISC-V Core

The core is a standard compliant RISC-V with 64 bit and integer implementation only. It is single cycle, without cache and memory hierarchy.

3.4.2 Instruction Memory

Instruction memory:

3.4.3 Data Memory

Data memory:

3.4.4 Wishbone Peripherals

GPIO

There are 8 GPIO pins implemented. Together with the data, an address will be given to the pins. With this, 16 times 8 GPIOs can be driven with just one external decoder.

3.4.5 Debug

The test and debug will be done via a JTAG interface. Until now, the loading of the instruction memory is possible.

Chapter 4

Architecture Concepts

The logic style used here is as often as possible a "one pulse" logic.

Figure 4.1 shows the "pulsed" logic.

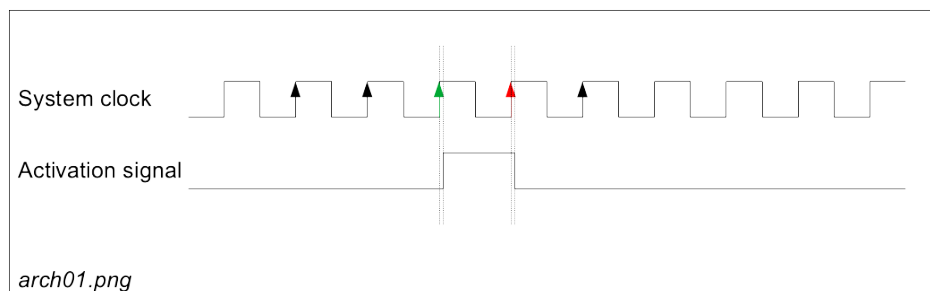


Figure 4.1: Enable with a Pulse

An "activation signal" will be generated synchronously with the "system clock" by its rising edge (green). This results in a delayed generation of the rising edge of the "activation signal". With the next rising edge of the "system clock" (red), the "activation signal" will be removed. Also here, the falling edge of the "activation signal" will have a small delay to the clock. So, the "activation signal" is exactly on "system clock" period high. This pair of signals can now be taken to start a following device (e.g. a counter), the counter will count only when the activation signal is high and the system clock will show a rising edge (red).

4.1 Technology

Table 4.1: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.2 CPU Concept

Table 4.2: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

The **RWU-RV64I** is a single cycle implementation without pipeline, memory hierarchy, MMU, etc. It is a students test chip.

Table 4.3 shows the notations which are used in the explanations for the instructions in tables 4.4 and 4.5.

Table 4.3: Notations for Instructions

Notation	Description
pc	program counter
rd	integer register destination
rsN	integer register source N
imm	immediate operand value
offs	immediate program counter relative offset
ux(reg)	unsigned XLEN-bit integer (32-bit on RV32, 64-bit on RV64)
sx(reg)	signed XLEN-bit integer (32-bit on RV32, 64-bit on RV64)
uN(reg)	zero extended N-bit integer register value
sN(reg)	sign extended N-bit integer register value
uN[reg + imm]	unsigned N-bit memory reference
sN[reg + imm]	signed N-bit memory reference

Table 4.4 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Table 4.4: RV32I Base Integer Instruction Set [2], [3], [1]

Instruction	Example	Meaning
Load Upper Immediate	<i>lui rd, imm</i>	$rd \leftarrow imm$
Add Upper Immediate to PC	<i>auipc rd, offs</i>	$rd \leftarrow pc + offs$
Jump and Link	<i>jal rd, offs</i>	$rd \leftarrow pc + len(instr),$ $pc \leftarrow pc + offs$
Jump and Link Register	<i>jalr rd, rs1, offs</i>	$rd \leftarrow pc + len(instr),$ $pc \leftarrow (rs1 + offs) \wedge -2$
Branch Equal	<i>beq rs1, rs2, offs</i>	<i>if</i> $rs1 = rs2$ <i>then</i> $pc \leftarrow pc + offs$
Branch Not Equal	<i>bne rs1, rs2, offs</i>	<i>if</i> $rs1 \neq rs2$ <i>then</i> $pc \leftarrow pc + offs$
Branch Less Than	<i>blt rs1, rs2, offs</i>	<i>if</i> $rs1 < rs2$ <i>then</i> $pc \leftarrow pc + offs$
Branch Greater than Equal	<i>bge rs1, rs2, offs</i>	<i>if</i> $rs1 \geq rs2$ <i>then</i> $pc \leftarrow pc + offs$

... next page

Instruction	Example	Meaning
Branch Less Than Unsigned	<i>bltu rs1, rs2, offs</i>	<i>if</i> $rs1 < rs2$ <i>then</i> $pc \leftarrow pc + offs$
Branch Greater than Equal Unsigned	<i>bgeu rs1, rs2, offs</i>	<i>if</i> $rs1 \geq rs2$ <i>then</i> $pc \leftarrow pc + offs$
Load Byte	<i>lb rd, offs(rs1)</i>	$rd \leftarrow s8[rs1 + offs]$
Load Half	<i>lh rd, offs(rs1)</i>	$rd \leftarrow s16[rs1 + offs]$
Load Word	<i>lw rd, offs(rs1)</i>	$rd \leftarrow s32[rs1 + offs]$
Load Byte Unsigned	<i>lbu rd, offs(rs1)</i>	$rd \leftarrow s8[rs1 + offs]$
Load Half Unsigned	<i>lhu rd, offs(rs1)</i>	$rd \leftarrow s16[rs1 + offs]$
Store Byte	<i>sb rs2, offs(rs1)</i>	$u8[rs1 + offs] \leftarrow rs2$
Store Half	<i>sh rs2, offs(rs1)</i>	$u16[rs1 + offs] \leftarrow rs2$
Store Word	<i>sw rs2, offs(rs1)</i>	$u32[rs1 + offs] \leftarrow rs2$
Add Immediate	<i>addi rd, rs1, imm</i>	$rd \leftarrow rs1 + sx(imm)$
Set Less Than Immediate	<i>slti rd, rs1, imm</i>	$rd \leftarrow sx(rs1) < sx(imm)$
Set Less Than Immediate Unsigned	<i>sltiu rd, rs1, imm</i>	$rd \leftarrow ux(rs1) < ux(imm)$
Xor Immediate	<i>xori rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \oplus ux(imm)$
Or Immediate	<i>ori rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \vee ux(imm)$
And Immediate	<i>andi rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \wedge ux(imm)$
Shift Left Logical Immediate	<i>slli rd, rs1, imm</i>	$rd \leftarrow ux(rs1) << ux(imm)$
Shift Right Logical Immediate	<i>srli rd, rs1, imm</i>	$rd \leftarrow ux(rs1) >> ux(imm)$
Shift Right Arithmetic Immediate	<i>srai rd, rs1, imm</i>	$rd \leftarrow sx(rs1) >> ux(imm)$
Add	<i>add rd, rs1, rs2</i>	$rd \leftarrow sx(rs1) + sx(rs2)$
Subtract	<i>sub rd, rs1, rs2</i>	$rd \leftarrow sx(rs1) - sx(rs2)$
Shift Left Logical	<i>sll rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) << rs2$
Set Less Than	<i>slt rd, rs1, rs2</i>	$rd \leftarrow sx(rs1) < sx(rs2)$
Set Less Than Unsigned	<i>sltu rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) < ux(rs2)$
Xor	<i>xor rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) \oplus ux(rs2)$
Shift Right Logical	<i>srl rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) >> rs2$
Shift Right Arithmetic	<i>sra rd, rs1, rs2</i>	$rd \leftarrow sx(rs1) >> rs2$
Or	<i>or rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) \vee ux(rs2)$

... next page

Instruction	Example	Meaning
And	<i>and rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) \wedge ux(rs2)$
Fence	<i>fence pred, succ</i>	
Fence Instruction	<i>fence.i</i>	

End of table.

Table 4.5 lists all base integer instructions of a RISC-V 64 bit architecture, additional to the 32 bit integer instruction set, according the specification [1].

Table 4.5: RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]

Instruction	Example	Meaning
Load Word Unsigned	<i>lwu rd, offs(rs1)</i>	$rd \leftarrow u32[rs1 + offs]$
Load Double	<i>ld rd, offs(rs1)</i>	$rd \leftarrow u64[rs1 + offs]$
Store Double	<i>sd rs2, offs(rs1)</i>	$u64[rs1 + offs] \leftarrow rs2$
Shift Left Logical Immediate	<i>slli rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \ll sx(imm)$
Shift Right Logical Immediate	<i>srli rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \gg sx(imm)$
Shift Right Arithmetic Immediate	<i>srai rd, rs1, imm</i>	$rd \leftarrow sx(rs1) \gg sx(imm)$
Add Immediate Word	<i>addiw rd, rs1, imm</i>	$rd \leftarrow s32(rs1) + imm$
Shift Left Logical Immediate Word	<i>slliw rd, rs1, imm</i>	$rd \leftarrow s32(u32(rs1) \ll imm)$
Shift Right Logical Immediate Word	<i>srliw rd, rs1, imm</i>	$rd \leftarrow s32(u32(rs1) \gg imm)$
Shift Right Arithmetic Immediate Word	<i>sraiw rd, rs1, imm</i>	$rd \leftarrow s32(rs1) \gg imm$
Add Word	<i>addw rd, rs1, rs2</i>	$rd \leftarrow s32(rs1) + s32(rs2)$
Subtract Word	<i>subw rd, rs1, rs2</i>	$rd \leftarrow s32(rs1) - s32(rs2)$
Shift Left Logical Word	<i>sllw rd, rs1, rs2</i>	$rd \leftarrow s32(u32(rs1) \ll rs2)$
Shift Right Logical Word	<i>srlw rd, rs1, rs2</i>	$rd \leftarrow s32(u32(rs1) \gg rs2)$

... next page

Instruction	Example	Meaning
Shift Right Arithmetic Word	<i>sraw rd, rs1, rs2</i>	$rd \leftarrow s32(rs1) \gg rs2$

End of table.

4.3 Bus Concept

Table 4.6: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22 2026-02-19	First draft (AS) First fill in cooperation with Claude-AI (AS)

4.3.1 Overview

The on-chip bus of this device is based on the **Wishbone B4 Classic** interconnect standard **wishboneB4**. Wishbone is a free, open-source hardware bus specification that defines a lightweight, synchronous point-to-point or shared bus protocol. It was chosen for this design because of its simplicity, its broad adoption in open-source IP cores, and its compatibility with the target RISC-V software ecosystem.

All peripherals and memory-mapped resources in this device are connected to the processor via a single shared Wishbone bus. Masters (currently: the RISC-V CPU core) initiate transactions; slaves (peripherals, memories) respond to them. The glue between each IP block and the bus is provided by a **Bus Peripheral Interface (BPI)** module that is instantiated once per master or slave.

Table 4.7 gives an overview of the bus parameters.

Table 4.7: Wishbone Bus Parameters

Parameter	Value	Comment
Standard	Wishbone B4 Classic	Single read/write cycles only
Data width	64 bit	One 64-bit word per transaction
Address width	64 bit	Full 64-bit address space
Byte select	8 bit	One bit per byte lane; this implementation requires all bits set
Wait states	0 (combinatorial ACK)	Slave must respond in the same clock cycle as STB
Clock domain	Single (bus clock)	All masters and slaves share one clock domain
Burst mode	Not supported	Each transaction is an independent single transfer
Error signal	Not implemented	ERR_O is not driven; bus errors are not signalled
Arbitration	Fixed priority	Currently one master only; arbitration is a future extension

4.3.2 Wishbone B4 Classic Protocol

Signal Naming Convention

Wishbone signal names in this design follow the convention shown in Table 4.8. The suffix `_i` indicates an input, `_o` an output, and `_s` an internal wire. The prefix `wb_s_` is used for slave-side signals and `wb_m_` for master-side signals.

Table 4.8: Wishbone Signal Naming

Wishbone Name	Slave Port	Master Port	Width	Description
ADR	wb_s_addr_i	wb_m_addr_o	64	Address bus
DAT_I	wb_s_dat_i	wb_m_dat_i	64	Data from master to slave (write data)
DAT_O	wb_s_dat_o	wb_m_dat_o	64	Data from slave to master (read data)
WE	wb_s_we_i	wb_m_we_o	1	Write enable: 1 = write, 0 = read
SEL	wb_s_sel_i	wb_m_sel_o	8	Byte select (all bytes must be set)
STB	wb_s_stb_i	wb_m_stb_o	1	Strobe: valid cycle indicator from master
ACK	wb_s_ack_o	wb_m_ack_i	1	Acknowledge: normal end of transfer from slave
CYC	wb_s_cyc_i	wb_m_cyc_o	1	Cycle: bus cycle active (held for complete transfer)

Transfer Protocol

A Wishbone Classic single transfer proceeds as follows:

1. The master asserts CYC and STB, presents a valid address on ADR, drives WE (1 for write, 0 for read), sets SEL to all-ones, and – for a write – provides data on DAT_O.

2. In this implementation the slave responds combinatorially in the same clock cycle: it drives read data on `DAT_O` and asserts `ACK`.
3. The transaction is complete when both `STB` and `ACK` are seen high on the rising clock edge. The master may then begin a new transfer in the very next cycle.

A transfer is considered valid only when both `CYC` and `STB` are asserted simultaneously and all byte-select bits are set. If any byte-select bit is clear the BPI treats the access as invalid and suppresses both the `ACK` and the write enable to the peripheral.

The timing diagram below illustrates a write followed by a read.

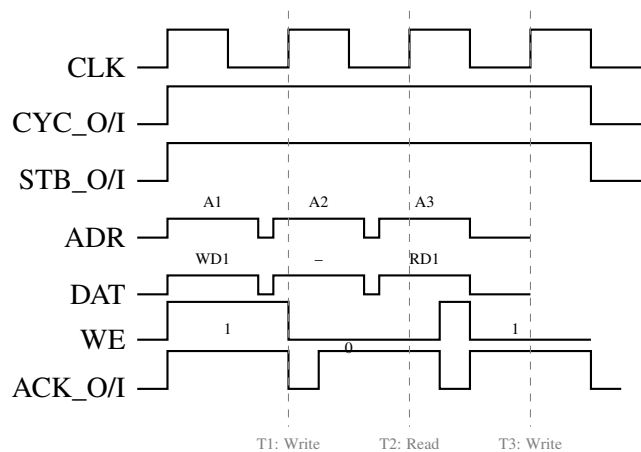


Figure 4.2: Wishbone Classic Single Transfers: Write – Read – Write

Byte Select

The 8-bit `SEL` signal indicates which byte lanes carry valid data. Bit n of `SEL` corresponds to byte lane n (bits $8n + 7 \dots 8n$) of the 64-bit data bus. In the current implementation, the master BPI always drives all eight `SEL` bits to 1 (full-word transfers only). The slave BPI checks that all `SEL` bits are set before asserting `ACK` or issuing a write enable to the peripheral kernel. Sub-word accesses are therefore not supported.

Reset Behaviour

All Wishbone outputs are driven to their inactive state while the synchronous active-high reset `rst_i` is asserted. For the master BPI this means `STB_O` and `CYC_O` are held low; for the slave BPI `ACK_O` is held low. Note that in the current

purely combinatorial implementation the reset is not registered inside the BPI modules; the reset is applied by the surrounding peripheral logic.

4.3.3 Wishbone Slave BPI (`as_slave_bpi`)

History

Table 4.9: History of `as_slave_bpi`

Version	Date	Author	Change
0.1	2026-02-19	Claude-AI	Initial version, Wishbone B4 Classic, zero wait states

Functional Configuration of `as_slave_bpi`

The `as_slave_bpi` module implements the Wishbone B4 Classic slave interface. It converts Wishbone bus signals into simple register-file read and write enables for the peripheral kernel. Table 4.10 lists the configurable parameters.

Table 4.10: Functional Configuration of `as_slave_bpi`

Parameter	Type	Default	Description
<code>addr_width</code>	integer	64	Width of the address bus in bits
<code>data_width</code>	integer	64	Width of the data bus in bits; also determines SEL width as <code>data_width/8</code>

HW Interfaces

Table 4.11 lists all input and output ports of the `as_slave_bpi` module.

Table 4.11: I/O of as_slave_bpi

Port	Direction	Width	Description
rst_i	in	1	Synchronous reset, active high
clk_i	in	1	Bus clock
Peripheral kernel side:			
addr_o	out	64	Decoded register address to peripheral
dat_from_core_i	in	64	Read data returned by peripheral kernel
dat_to_core_o	out	64	Write data forwarded to peripheral kernel
wr_o	out	1	Write enable to peripheral: asserted for one cycle on a valid write
rd_o	out	1	Read enable to peripheral: asserted for one cycle on a valid read
Wishbone bus side:			
wb_s_addr_i	in	64	Address from Wishbone master
wb_s_dat_i	in	64	Write data from Wishbone master
wb_s_dat_o	out	64	Read data to Wishbone master
wb_s_we_i	in	1	Write enable from Wishbone master
wb_s_sel_i	in	8	Byte select from Wishbone master
wb_s_stb_i	in	1	Strobe from Wishbone master
wb_s_ack_o	out	1	Acknowledge to Wishbone master
wb_s_cyc_i	in	1	Cycle from Wishbone master

Bus Interface

Wishbone Classic Slave.

Registers

The `as_slave_bpi` module contains no registers of its own. All logic is purely combinatorial. Registers are located in the peripheral kernel that is connected via the kernel-side interface of this module.

Related Sections of this Specification

- Wishbone Master BPI Section 4.3.4
- QSPI Peripheral Section 6.3, which uses this BPI
- GPIO Peripheral Section 6.1, which uses this BPI

Functional Overview

The slave BPI converts two Wishbone control signals plus the byte-select into two conditions used throughout the peripheral:

$$valid_transfer = CYC \wedge STB$$

$$all_sel = \& SEL_{7:0}$$

From these two conditions the write and read enables to the peripheral kernel are derived:

$$wr_o = valid_transfer \wedge WE \wedge all_sel$$

$$rd_o = valid_transfer \wedge \neg WE \wedge all_sel$$

The Wishbone acknowledge is generated combinatorially:

$$ACK_O = valid_transfer \wedge all_sel$$

Because ACK is driven combinatorially from STB and CYC, the slave introduces zero wait states. The peripheral kernel must therefore provide read data (`dat_from_core_i`) in the same clock cycle that `rd_o` is asserted.

Structural Overview

Figure 4.3 shows the internal signal flow of the slave BPI.

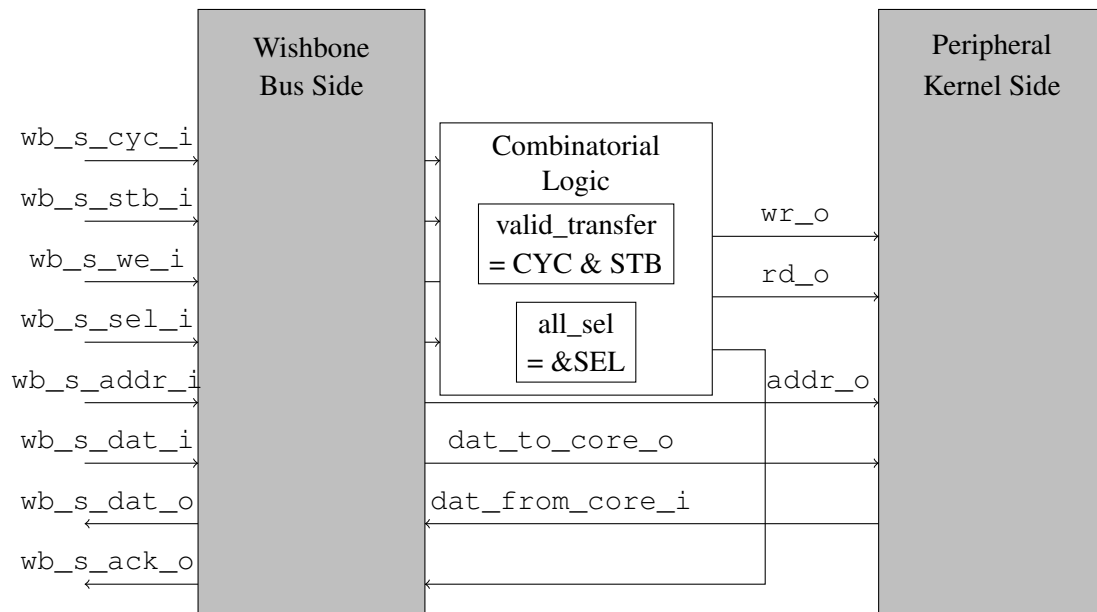


Figure 4.3: as_slave_bpi – Internal Signal Flow

Table 4.12 lists the internal signals.

Table 4.12: as_slave_bpi Internal Signals

Signal	Type	Width	Description
valid_transfer_s	logic	1	High when both CYC and STB are asserted. Indicates an active Wishbone bus cycle.
all_sel_s	logic	1	High when all byte-select bits are set. Only full 64-bit word accesses are accepted.

Instantiation

The module is parameterised and instantiated as follows:

```
as_slave_bpi #(
    .addr_width ( 64 ),
    .data_width ( 64 )
) u_qspi_bpi (
    .rst_i          ( rst_i          ),
    .clk_i          ( clk_i          ),
    // kernel side
```

```

.addr_o          ( addr_s          ),
.dat_from_core_i ( dat_from_core_s ),
.dat_to_core_o   ( dat_to_core_s   ),
.wr_o           ( wr_s            ),
.rd_o           ( rd_s            ),
// wishbone side
.wb_s_addr_i     ( wb_s_addr_i     ),
.wb_s_dat_i      ( wb_s_dat_i      ),
.wb_s_dat_o      ( wb_s_dat_o      ),
.wb_s_we_i       ( wb_s_we_i       ),
.wb_s_sel_i      ( wb_s_sel_i      ),
.wb_s_stb_i      ( wb_s_stb_i      ),
.wb_s_ack_o      ( wb_s_ack_o      ),
.wb_s_cyc_i      ( wb_s_cyc_i      )
);

```

Limitations and Design Rules

- **Zero wait states only.** The peripheral kernel must provide read data combinatorially. If the kernel uses registered outputs, a wait-state mechanism (delayed ACK) must be added externally.
- **Full-word accesses only.** Sub-word byte or half-word accesses are not supported. The SEL check ensures that partial accesses are rejected silently.
- **No ERR_O.** Bus errors are not signalled. Invalid accesses are simply not acknowledged (ACK stays low).
- **No burst support.** The module supports only Wishbone Classic single-cycle transfers. Pipeline or burst modes are not implemented.
- **Combinatorial ACK.** Connecting multiple slave BPIs in a shared-bus topology requires that ACK is only driven by the selected slave. Address decoding and ACK gating must be handled by the bus interconnect logic, not by this module.

4.3.4 Wishbone Master BPI (`as_master_bpi`)

History

Table 4.13: History of `as_master_bpi`

Version	Date	Author	Change
0.1	2026-02-19	Claude-AI	Initial version, Wishbone B4 Classic, continuous CYC/STB

Functional Configuration of `as_master_bpi`

The `as_master_bpi` module implements the Wishbone B4 Classic master interface. It is a simple combinatorial wrapper that translates address, data and direction signals from the CPU or DMA core into Wishbone master signals. Table 4.14 lists the configurable parameters.

Table 4.14: Functional Configuration of `as_master_bpi`

Parameter	Type	Default	Description
<code>addr_width</code>	integer	64	Width of the address bus in bits
<code>data_width</code>	integer	64	Width of the data bus in bits; also determines SEL width as <code>data_width/8</code>

HW Interfaces

Table 4.15 lists all input and output ports of the `as_master_bpi` module.

Table 4.15: I/O of `as_master_bpi`

Port	Direction	Width	Description
<code>rst_i</code>	in	1	Synchronous reset, active high
<code>clk_i</code>	in	1	Bus clock
CPU / master core side:			
<code>addr_i</code>	in	64	Address from CPU core
<code>dat_from_core_i</code>	in	64	Write data from CPU core
<code>dat_to_core_o</code>	out	64	Read data returned to CPU core
<code>wr_i</code>	in	1	Write enable from CPU core: 1 = write, 0 = read
Wishbone bus side:			
<code>wb_m_addr_o</code>	out	64	Address to Wishbone bus
<code>wb_m_dat_i</code>	in	64	Read data from Wishbone bus (from selected slave)
<code>wb_m_dat_o</code>	out	64	Write data to Wishbone bus
<code>wb_m_we_o</code>	out	1	Write enable to Wishbone bus
<code>wb_m_sel_o</code>	out	8	Byte select to Wishbone bus (always all-ones)
<code>wb_m_stb_o</code>	out	1	Strobe to Wishbone bus (tied high)
<code>wb_m_ack_i</code>	in	1	Acknowledge from Wishbone bus (from selected slave)
<code>wb_m_cyc_o</code>	out	1	Cycle to Wishbone bus (tied high)

Bus Interface

Wishbone Classic Master.

Registers

The `as_master_bpi` module contains no registers of its own. All logic is purely combinatorial.

Related Sections of this Specification

- Wishbone Slave BPI Section 4.3.3
- RWU-RV64I Core Chapter, Bus Interface subsection

Functional Overview

The master BPI is a thin combinatorial bridge. All Wishbone master outputs are driven directly from the CPU core signals without any registered state:

```
wb_m_addr_o = addr_i
wb_m_dat_o = dat_from_core_i
wb_m_we_o = wr_i
wb_m_sel_o = 8'hFF (all bytes always valid)
wb_m_stb_o = 1
wb_m_cyc_o = 1
dat_to_core_o = wb_m_dat_i
```

Because STB and CYC are continuously asserted, a new Wishbone transfer is initiated on every clock cycle as long as a slave is present on the bus. The transfer completes when the slave returns ACK. In the zero-wait-state case (slave BPI also combinatorial), ACK arrives in the same cycle as STB, so every clock cycle is a completed transfer.

The CPU core must handle the `ack_i` signal to determine when a read result is valid or when the next write can proceed.

Structural Overview

Figure 4.4 shows the internal signal flow of the master BPI.

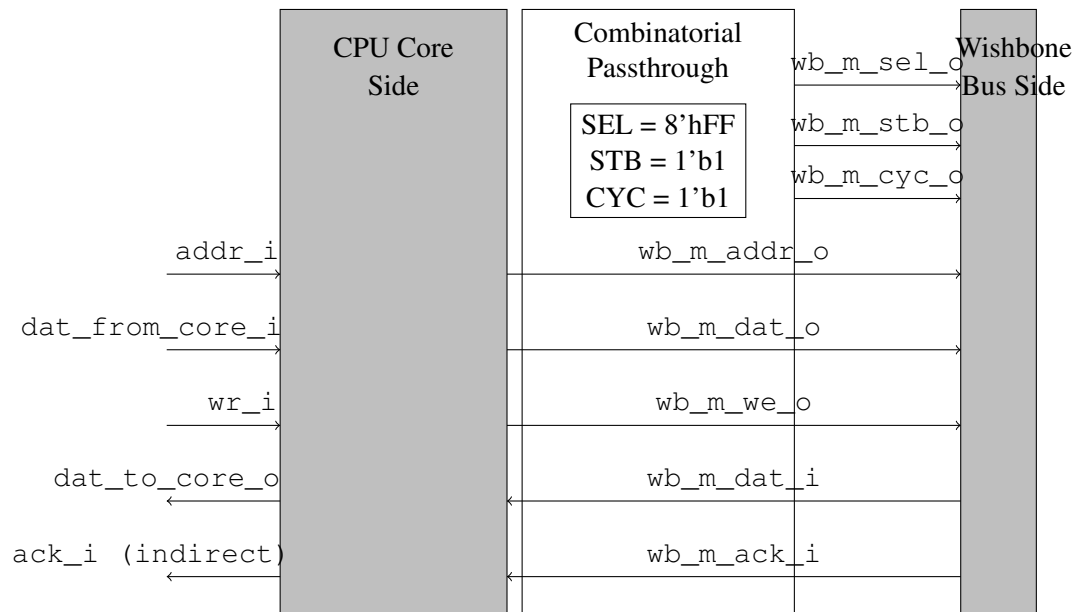


Figure 4.4: as_master_bpi – Internal Signal Flow

Table 4.16 lists the internal signals (none in this implementation, as the module is purely combinatorial):

Table 4.16: as_master_bpi Internal Signals

Signal	Type	Width	Description
(none)	–	–	All logic is combinatorial; no internal signals

Instantiation

```
as_master_bpi #(
    .addr_width ( 64 ),
    .data_width ( 64 )
) u_cpu_bpi (
    .rst_i           ( rst_i           ),
    .clk_i           ( clk_i           ),
    // CPU core side
    .addr_i          ( cpu_addr_s      ),
    .dat_from_core_i ( cpu_wdat_s      ),
    .dat_to_core_o    ( cpu_rdat_s      ),
    .wr_i            ( cpu_wr_s        ),
    // wishbone side
```



```

.wb_m_addr_o      ( wb_m_addr_s      ),
.wb_m_dat_i       ( wb_m_dat_i_s      ),
.wb_m_dat_o       ( wb_m_dat_o_s      ),
.wb_m_we_o        ( wb_m_we_s         ),
.wb_m_sel_o       ( wb_m_sel_s        ),
.wb_m_stb_o       ( wb_m_stb_s        ),
.wb_m_ack_i       ( wb_m_ack_s        ),
.wb_m_cyc_o       ( wb_m_cyc_s        )
);

```

Limitations and Design Rules

- **Continuous bus occupation.** Because STB and CYC are permanently asserted, the master occupies the bus at all times. Multi-master arbitration requires that these signals are gated by the arbiter before being placed on the shared bus.
- **No idle state.** The master BPI does not support idle cycles between transfers. The surrounding CPU core logic must insert idle cycles if needed by controlling `addr_i` and `wr_i` appropriately and ignoring any ACK responses during idle periods.
- **ACK not evaluated internally.** The `wb_m_ack_i` signal is passed through directly to the CPU core via the bus-side port; the BPI itself does not act on it. The CPU must sample ACK and stall its pipeline if required.
- **No error handling.** There is no support for Wishbone `ERR_I` or `RTY_I`.

4.3.5 Bus Interconnect

Address Map

Each slave is mapped to a contiguous, non-overlapping region of the 64-bit address space. The actual address boundaries are defined in the chip-level memory map (Chapter “Memory Map and Registers”).

Address decoding is performed by the bus interconnect logic, which forwards STB and CYC only to the addressed slave. Slaves whose address range is not selected must drive `ACK_O` low and `DAT_O` to zero.

Shared Bus vs. Crossbar

The current implementation uses a **shared bus** topology with a single master (the CPU core). A crossbar switch is not implemented because:

- There is currently only one bus master.
- The zero-wait-state slave BPI design means bus bandwidth is limited only by the clock frequency, not by bus contention.

If additional masters (e.g. a DMA controller) are added in the future, the bus interconnect must be extended to include an arbiter.

Peripheral Register Offset Alignment

All peripheral registers are 64 bit wide and aligned to 8-byte boundaries. The address offset within a peripheral starts at 0×00 for the ID register and increases in steps of 0×08 . This is consistent with the Wishbone bus data width and the 64-bit alignment of the RISC-V load and store instructions used to access the peripheral registers.

Read-Modify-Write

The Wishbone protocol as implemented does not support atomic read-modify-write operations. Software must therefore use disable-interrupt / critical-section patterns when accessing shared registers from multiple contexts.

4.3.6 Comparison: Slave BPI vs. Master BPI

Table 4.17 summarises the key differences between the two BPI modules.

Table 4.17: Slave BPI vs. Master BPI – Key Differences

Property	Slave BPI (as_slave_bpi)	Master BPI (as_master_bpi)
Role	Peripheral / memory side	CPU / initiator side
SEL handling	Checks all bits; rejects partial accesses	Always drives all-ones
STB / CYC	Receives from master; de-codes	Drives permanently high
ACK	Generates combinatorially	Receives; forwards to core
Internal state	None (combinatorial)	None (combinatorial)
Wait states	Zero (combinatorial ACK)	Accepts any number (stalls when ACK=0)
rd_o / wr_o	Provides to peripheral kernel	N/A (uses wr_i from core)
Synthesis result	≈ 5 LUTs	≈ 0 LUTs (wire-through)

4.4 Interrupt Concept

Table 4.18: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.5 Clock Concept

Table 4.19: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.6 System Control Concept

Table 4.20: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.7 Power Concept

Table 4.21: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.8 Memory Concept

Table 4.22: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.9 System Protection and Security Concept

Table 4.23: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.10 Debug Concept

Table 4.24: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.11 Register Concept

Table 4.25: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

Chapter 5

RWU-RV64I Core

5.1 ALU

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 5.1: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 5.2: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)

Functional Configuration of ALU0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a ALU peripheral are described in its functional description in Section 5.1.2. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for ALU0 in Table 5.3.

The **RWU-RV64I** has only one ALU.

Table 5.3: ALU0 Feature Set

	Generic GPIO Feature Set	Details
no	Seperate Kernel Clock	not needed
no	FIFO Data Buffering	not needed

HW Interfaces

The following table 5.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 5.4: HW Interfaces of Module ALU0

	Module internal name	Chip level name
Supply		
Clock	-	-
Bus	-	-
Interrupt	-	-
DMA	-	-
External Signals	-	-
Others	data01_i(63:0) data02_i(63:0) aluSel_i(4:0) aluZero_o aluNega_o aluCarr_o aluOver_o aluResult_o(63:0)	srcA_s(63:0) srcB_s(63:0) aluSel_s(4:0) zero_s nega_s carry_s overflow_s dBusAddr_s(63:0)

Bus Interface

None.

Registers

None.

Related Sections of this Spec

None.

5.1.1 History of ALU

Table 5.5: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
5.1	5.1 / 2025-02-13	First draft (AS)

5.1.2 Functional Overview

The ALU operates on 64 bit data and generates 64 bit data. The ALU indicates a zero result, a negative result, the carry of a result, and the overflow.

General Features:

- 64 bit integer addition
- 64 bit integer subtraction
- 64 bit AND
- 64 bit OR
- 64 bit XOR
- 64 bit set less than
- 64 bit set less than, unsigned
- 32 bit shift right arithmetic word
- 32 bit shift right logical word
- 32 bit shift left logical word
- 32 bit subtract word
- 32 bit add word

- 64 bit shift right arithmetic
- 64 bit shift right logical
- 64 bit shift left logical
- branch if equal
- branch if not equal
- branch if less than
- branch if greater or equal
- branch if less than, unsigned
- branch if greater or equal, unsigned

Synchronous Mode:

- -

Asynchronous Mode:

- -

5.1.3 Structural Overview**I/O of the ALU**

Table 5.6 shows the input and outputs of the ALU0. This are all primary inputs and outputs. There is no bi-directional, tristate or open drain pin. All pins are high-active except the pins with a `_n_` in its name.

Table 5.6: ALU0 I/O

Pin	Dir.	Wd.	Explanation
data01_i	in	64	Source data input. Comes either from the register file or the program counter (PC). Switched by signal “aluSrcA”. Source one for all ALU operators.
data02_i	in	64	Source data input. Comes either from the register file or the immediate generator. Switched by signal “aluSrcB”. Source two for all ALU operators.
aluSel_i	in	5	Selects the ALU operator.
aluZero_o	out	1	Zero flag of the ALU. Not buffered.
aluNega_o	out	1	Negative flag of the ALU. Not buffered.
aluCarr_o	out	1	Carry flag of the ALU. Not buffered.
aluOver_o	out	1	Overflow flag of the ALU. Not buffered.
aluResult_o	out	64	The result of each ALU operation.

Internals of the ALU

The following list (Table) shows the internal signals of the ALU. The table shows the signal name, the width, the driver, the target and a short explanation.

- Signal: aluSel_i[0]
 - Width: 1 bit
 - Source: aluSel_i
 - Destination: aluResult_o, overf_s
 - Explanation (special signal):
 - * if aluSel_i[0] = 0, it is an ADD instruction
 - * ... the second operand must be taken as it is
 - * ... the carry-in of the addition is 0 (aluSel_i[0])
 - * if aluSel_i[0] = 1, it is a SUB instruction
 - * ... the second operand must be the 2-complement
 - * ... the carry-in of the addition is 1 which acts as the +1 (aluSel_i[0])
- Signal: data01_s
 - Width: 64 bit, signed
 - Source: data01_i
 - Destination: aluResult_o, aluZero_o

- Explanation: It is the signed interpretation of `data01_i`. It is needed for some branches (BLT, BGE) and SRA.

logic signed [reg_width-1:0] data01_s;

- Signal: `data02_s`

- Width: 64 bit, signed
- Source: `data02_i`
- Destination: `aluResult_o`, `aluZero_o`
- Explanation: It is the signed interpretation of `data02_i`. It is needed for some branches (BLT, BGE) and SRA.

logic signed [reg_width-1:0] data02_s;

- Signal: `sum_s`

- Width: 64 bit
- Source: `data01_i`, `cinvb_s` (`data02_i`), `aluSel_i[0]`
- Destination: `aluResult_o`
- Explanation: It is the sum of the two data inputs `data01_i` and `data02_i`. The second data input could be negative and so 2-complemented. `cinvb_s` is the inversion of `data02_e` when `aluSel_i[0] = 1`, `aluSel_i[0]` also acts as the plus 1.

assign {carry_s, sum_s} = data01_i + cinvb_s + aluSel_i[0];

- Signal: `cinvb_s`

- Width: 64 bit
- Source: `data02_i`, `aluSel_i[0]`
- Destination: `sum_s`, `carry_s`
- Explanation: `cinvb_s` is the inversion of `data02_i` when `aluSel_i[0] = 1`. Together with `aluSel_i[0]` (`cinvb_s + aluSel_i[0]`) it is the 2-complement of `data02_i`.

assign cinvb_s = aluSel_i[0] ? data02_i : data02_i;

- Signal: `carry_s`

- Width: 1 bit
- Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
- Destination: aluCarr_o (carry flag)
- Explanation: It is the carry of sum_s.

assign {carry_s, sum_s} = data01_i + cinvb_s + aluSel_i[0];

- Signal: sumw_s

- Width: 32 bit
- Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
- Destination: aluResult_o
- Explanation: For add word, etc. It is the sum of the two data inputs data01_i(31:0) and data02_i(31:0). The second data input could be negative and so 2-complemented. cinvbw_s is the inversion of data02_e(31:0) when *aluSel_i*[0] = 1, aluSel_i[0] also acts as the plus 1.

assign {carryw_s, sumw_s} = data01_i[31:0] + cinvbw_s + aluSel_i[0];

- Signal: cinvbw_s

- Width: 32 bit
- Source: data02_e, aluSel_i[0]
- Destination: sum_s, carry_s
- Explanation: For add word, etc. cinvbw_s is the inversion of data02_i(31:0) when *aluSel_i*[0] = 1. Together with aluSel_i[0] (*cinvbw_s*+*aluSel_i*[0]) it is the 2-complement of data02_i(31:0).

assign cinvbw_s = aluSel_i[0] ? data02_i[31:0] : data02_i[31:0];

- Signal: carryw_s

- Width: 1 bit
- Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
- Destination: aluCarr_o (carry flag)
- Explanation: For add word, etc. It is the carry of sumw_s.

assign {carryw_s, sumw_s} = data01_i[31:0] + cinvbw_s + aluSel_i[0];

- Signal: `sllw_s`
 - Width: 32 bit
 - Source: `data01_i`, `data02_i`
 - Destination: `aluResult_o`
 - Explanation: SLLW (shift left logical word)
Implementation:
 $sllw_s = data01_i(31 : 0) \ll data02_i(4 : 0)$
`aluResult_o` is `sllw_s`, sign extended.
- `srlw_s`
 - Width: 32 bit
 - Source: `data01_i`, `data02_i`
 - Destination: `aluResult_o`
 - Explanation: SRLW (shift right logical word)
Implementation:
 $srlw_s = data01_i(31 : 0) \gg data02_i(4 : 0)$
`aluResult_o` is `srlw_s`, sign extended.
- `sraw_s`
 - Width: 32 bit, signed
 - Source: `data01_i`, `data02_i`
 - Destination: `aluResult_o`
 - Explanation: SRAW (shift right algorithmic word)
Implementation:
 $sraw_s = data01_i(31 : 0) \ggg data02_i(4 : 0)$
`aluResult_o` is `sraw_s`, sign extended.
- `data01w_s`
 - Width: 32 bit, signed
 - Source: `data01_i`
 - Destination: `sraw_s`
 - Explanation: 32 bit signed representation of `data01_i`.
- `sltiu_s`
 - Width: 1 bit

- Source: data01_i, data02_i
- Destination: aluResult_o
- Explanation: SLT, SLTU (set less than; signed, unsigned)
Implementation:
if(data01_i < data02_i)
 sltiu_s = 1;
else
 sltiu_s = 0;
aluResult_o is sltiu_s, zero extended.
- overf_s
 - Width: 1 bit
 - Source: data01_i, data02_i, sum_s, aluSel_i[0]
 - Destination: aluOver_o
 - Explanation: Overflow occurs when the result-value affects the sign:
 - overflow when adding two positives yields a negative
 - or, adding two negatives gives a positive
 - or, subtract a negative from a positive and get a negative
 - or, subtract a positive from a negative and get a positive
- aluResult_o
 - Width: 64 bit
 - Source: all signals
 - Destination: output
 - Explanation: The result of each ALU operation.
If aluSel_i =

0	aluResult_o = sum_s;	ADD, ADDI, LB, LH, LW, LBU, LHU, LD, LWU, AUIPC, LUI, SB, SH, SW, SD, JALR, JAL
1	aluResult_o = sum_s;	SUB
2	aluResult_o = data01_i & data02_i;	AND, ANDI
3		OR
4		XOR
5		SLT
6		SLTU

5.1.4 Service Requests of the ALU

None.

5.1.5 The ALU Peripheral Kernel

The ALU calculates a result (aluResult_o) and the corresponding flags zero (aluZero_o), negative (aluNega_o), carry (asCarr_o) and overflow (asOver_o). Table 5.7 and table 5.8 are describing the function of the ALU for each instruction.

The zero flag is not only a zero flag but also the control for all branch instruction.

Table 5.7 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Table 5.7: RV32I Base Integer Instruction Set [2], [3], [1]

Instruction	Operation	Remarks
Load Upper Immediate	-	-
Add Upper Immediate to PC	-	-
Jump and Link	$jal\ rd, offs$	$rd \leftarrow pc + len(instr),$ $pc \leftarrow pc + offs$
Jump and Link Register	$jalr\ rd, rs1, offs$	$rd \leftarrow pc + len(instr),$ $pc \leftarrow (rs1 + offs) \wedge -2$
Branch Equal	$beq\ rs1, rs2, offs$	$if\ rs1 = rs2\ then\ pc \leftarrow pc + offs$
Branch Not Equal	$bne\ rs1, rs2, offs$	$if\ rs1 \neq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than	$blt\ rs1, rs2, offs$	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal	$bge\ rs1, rs2, offs$	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than Unsigned	$bltu\ rs1, rs2, offs$	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal Unsigned	$bgeu\ rs1, rs2, offs$	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Load Byte	$lb\ rd, offs(rs1)$	$rd \leftarrow s8[rs1 + offs]$
Load Half	$lh\ rd, offs(rs1)$	$rd \leftarrow s16[rs1 + offs]$
Load Word	$lw\ rd, offs(rs1)$	$rd \leftarrow s32[rs1 + offs]$

... next page

Instruction	Operation	Remarks
Load Byte Unsigned	<i>lbu rd, offs(rs1)</i>	$rd \leftarrow s8[rs1 + offs]$
Load Half Unsigned	<i>lhu rd, offs(rs1)</i>	$rd \leftarrow s16[rs1 + offs]$
Store Byte	<i>sb rs2, offs(rs1)</i>	$u8[rs1 + offs] \leftarrow rs2$
Store Half	<i>sh rs2, offs(rs1)</i>	$u16[rs1 + offs] \leftarrow rs2$
Store Word	<i>sw rs2, offs(rs1)</i>	$u32[rs1 + offs] \leftarrow rs2$
Add Immediate	<i>addi rd, rs1, imm</i>	$rd \leftarrow rs1 + sx(imm)$
Set Less Than Immediate	<i>slti rd, rs1, imm</i>	$rd \leftarrow sx(rs1) < sx(imm)$
Set Less Than Immediate Unsigned	<i>sltiu rd, rs1, imm</i>	$rd \leftarrow ux(rs1) < ux(imm)$
Xor Immediate	<i>xori rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \oplus ux(imm)$
Or Immediate	<i>ori rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \vee ux(imm)$
And Immediate	<i>andi rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \wedge ux(imm)$
Shift Left Logical Immediate	<i>slli rd, rs1, imm</i>	$rd \leftarrow ux(rs1) << ux(imm)$
Shift Right Logical Immediate	<i>srli rd, rs1, imm</i>	$rd \leftarrow ux(rs1) >> ux(imm)$
Shift Right Arithmetic Immediate	<i>srai rd, rs1, imm</i>	$rd \leftarrow sx(rs1) >> ux(imm)$
Add	aluResult_o = sum_s = data01_i + cinvb_s + aluSel_i[0]	aluSel_i = 0; cinvb_s = aluSel_i[0] ? not data02_i : data02_i
Subtract	<i>sub rd, rs1, rs2</i>	$rd \leftarrow sx(rs1) - sx(rs2)$
Shift Left Logical	<i>sll rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) << rs2$
Set Less Than	<i>slt rd, rs1, rs2</i>	$rd \leftarrow sx(rs1) < sx(rs2)$
Set Less Than Unsigned	<i>sltu rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) < ux(rs2)$
Xor	<i>xor rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) \oplus ux(rs2)$
Shift Right Logical	<i>srl rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) >> rs2$
Shift Right Arithmetic	<i>sra rd, rs1, rs2</i>	$rd \leftarrow sx(rs1) >> rs2$
Or	<i>or rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) \vee ux(rs2)$
And	<i>and rd, rs1, rs2</i>	$rd \leftarrow ux(rs1) \wedge ux(rs2)$
Fence	<i>fence pred, succ</i>	
Fence Instruction	<i>fence.i</i>	

End of table.

Table 5.8 lists all base integer instructions of a RISC-V 64 bit architecture, additional to the 32 bit integer instruction set, according the specification [1].

Table 5.8: RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]

Instruction	Example	Meaning
Load Word Unsigned	<i>lwu rd, offs(rs1)</i>	$rd \leftarrow u32[rs1 + offs]$
Load Double	<i>ld rd, offs(rs1)</i>	$rd \leftarrow u64[rs1 + offs]$
Store Double	<i>sd rs2, offs(rs1)</i>	$u64[rs1 + offs] \leftarrow rs2$
Shift Left Logical Immediate	<i>slli rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \ll sx(imm)$
Shift Right Logical Immediate	<i>srlr rd, rs1, imm</i>	$rd \leftarrow ux(rs1) \gg sx(imm)$
Shift Right Arithmetic Immediate	<i>srai rd, rs1, imm</i>	$rd \leftarrow sx(rs1) \gg sx(imm)$
Add Immediate Word	<i>addiw rd, rs1, imm</i>	$rd \leftarrow s32(rs1) + imm$
Shift Left Logical Immediate Word	<i>slliw rd, rs1, imm</i>	$rd \leftarrow s32(u32(rs1) \ll imm)$
Shift Right Logical Immediate Word	<i>srliw rd, rs1, imm</i>	$rd \leftarrow s32(u32(rs1) \gg imm)$
Shift Right Arithmetic Immediate Word	<i>sraiw rd, rs1, imm</i>	$rd \leftarrow s32(rs1) \gg imm$
Add Word	<i>addw rd, rs1, rs2</i>	$rd \leftarrow s32(rs1) + s32(rs2)$
Subtract Word	<i>subw rd, rs1, rs2</i>	$rd \leftarrow s32(rs1) - s32(rs2)$
Shift Left Logical Word	<i>sllw rd, rs1, rs2</i>	$rd \leftarrow s32(u32(rs1) \ll rs2)$
Shift Right Logical Word	<i>srlw rd, rs1, rs2</i>	$rd \leftarrow s32(u32(rs1) \gg rs2)$
Shift Right Arithmetic Word	<i>sraw rd, rs1, rs2</i>	$rd \leftarrow s32(rs1) \gg rs2$

End of table.

5.1.6 Registers

None.

5.2 Instruction Decoder

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 5.9: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 5.10: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)

Functional Configuration of InstructionDecode0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a Instruction Decoder are described in its functional description in Section 5.2.2. Generic features, that can be decided for each specific

instantiation of the peripheral are listed and completed for InstructionDecode0 in Table 5.11.

The **RWU-RV64I** has only one Instruction Decoder.

Table 5.11: InstrDec0 Feature Set

	Generic GPIO Feature Set	Details
no	Seperate Kernel Clock	not needed
no	FIFO Data Buffering	not needed

HW Interfaces

The following table 5.12 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 5.12: HW Interfaces of Module InstrDec0

	Module internal name	Chip level name
Supply		
Clock	-	-
Bus	-	-
Interrupt	-	-
DMA	-	-
External Signals	-	-
Others	opcode_i(6:0) func3_i(2:0) func7b5_i zero_i resultSrc_o(1:0) dMemWr_o dMemRd_o PCSrc_o aluSrcB_o aluSrcA_o regWr_o jump_o immSrc_o(2:0) aluSel_o(4:0) branch_s_o jumo_s_o	iBusDataRd_s(6:0) iBusDataRd_s(14:12) iBusDataRd_s(30) zero_s resultSrc_s(1:0) dMemWr_s dMemRd_s PCSrc_s (not for pipeline) aluSrcB_s aluSrcA_s regWr_s jump_s immSrc_s(2:0) aluSel_s(4:0) only for pipeline only for pipeline

Bus Interface

None.

Registers

None.

Related Sections of this Spec

None.

5.2.1 History of Instruction Decoder

Table 5.13: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
5.2	5.2 / 2025-02-13	First draft (AS)

5.2.2 Functional Overview

The Instruction Decoder operates on 32 bit instructions and generates all control signals needed for the operation of the CPU.

General Features:

- Decodes all instructions of the RV64I:
 - opcode=3: I-type, loads
 - opcode=19: I-type, immediate arithmetics, logic and shift
 - opcode=23: U-type, auipc
 - opcode=27: I-type, immediate arithmetics, logic and shift (word)
 - opcode=35: S-type, stores
 - opcode=51: R-type, RV32I
 - opcode=55: U-type, lui
 - opcode=59: R-type, RV64I
 - opcode=99: B-type, all conditional branches
 - opcode=103: I-type, jalr
 - opcode=111: J-type, jal
- generates the control signal for the ALU,
- ... for the D-Mem MUX,
- ... for the D-Mem read- and write enable,

- ... for the PC MUX,
- ... for the register A and B output MUXes,
- ... for the register file write enable,
- ... for the branch control,
- ... for the immediate generation selection,
- ... for the jump and branch control in the pipelined version.

Synchronous Mode:

- -

Asynchronous Mode:

- -

5.2.3 Structural Overview

I/O of the Instruction Decoder

Table 5.14 shows the input and outputs of the ALU0. This are all primary inputs and outputs. There is no bi-directional, tristate or open drain pin. All pins are high-active except the pins with a `_n_` in its name.

Table 5.14: InstrDec0 I/O

Pin	Dir.	Wd.	Explanation
opcode_i	in	7	The opcode field of the instruction. This are the least significant 7 bits of the instruction.
func3_i	in	3	The func3-field of the instruction. It specifies the opcode.
func7b5_i	in	1	Bit 5 of func7 field.
zero_i	in	1	Indicates a branch. Comes from the ALU.
resultSrc_o	out	2	Controls the Mux behind the DMem.
dMemWr_o	out	1	D-Mem write enable.
dMemRd_o	out	1	D-Mem read enable; almost not needed anymore.
PCSrc_o	out	1	Control of the Mux for the PC; normal operation or jump/branch.
aluSrcB_o	out	1	Data Mux in front of ALU.
aluSrcA_o	out	1	Data Mux in front of ALU.
regWr_o	out	1	Register file write enable.
jump_o	out	1	Src for branch target mux: register or PC as input for branch target adder.
immSrc_o	out	3	Selects, how the immediate value will be generated.
aluSel_o	out	5	ALU control.
branch_s_o	out	1	only for pipeline solution
jump_s_o	out	1	only for pipeline solution

Internals of the Instruction Decoder

The following list (Table) shows the internal signals of the Instruction Decoder. The table shows the signal name, the width, the driver, the target and a short explanation.

- Signal: opcode_i
 - Width: 7 bit unsigned
 - Source: Instruction from I-Mem.
 - Destination: Case statement generating controls_s and aluSel_o
 - Explanation:
 - * opcode=3: loads
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'b000; Instruction format: I-type
 - aluSrcB_o=1; Mux - Second register or immediate: immediate

- aluSrcA_o=0; Mux - First register or PC: register
- dMemWr_o=0; Write to D-Mem: No
- dMemRd_o=1; Read from D-Mem: Yes
- resultSrc_o=2'b01; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: D-Mem-data to RegFile
- branch_s=0; Branch: No
- jump_s=0; Only for jalr, jal
- aluSel_o=5'b00000 (ADD)
- * opcode=19: arithmetic immediate
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'b000; Instruction format: I-type
 - aluSrcB_o=1; Mux - Second register or immediate: immediate
 - aluSrcA_o=0; Mux - First register or PC: register
 - dMemWr_o=0; Write to D-Mem: No
 - dMemRd_o=0; Read from D-Mem: No
 - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
 - branch_s=0; Branch: No
 - jump_s=0; Only for jalr, jal
 - aluSel_o=various
- * opcode=23: auipc
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'b100; Instruction format: U-type
 - aluSrcB_o=1; Mux - Second register or immediate: immediate
 - aluSrcA_o=1; Mux - First register or PC: PC
 - dMemWr_o=0; Write to D-Mem: No
 - dMemRd_o=0; Read from D-Mem: No
 - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
 - branch_s=0; Branch: No
 - jump_s=0; Only for jalr, jal
 - aluSel_o=5'b00000 (ADD)
- * opcode=27: arithmetic immediate (64 bit)
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'b000; Instruction format: I-type
 - aluSrcB_o=1; Mux - Second register or immediate: immediate

- aluSrcA_o=0; Mux - First register or PC: register
- dMemWr_o=0; Write to D-Mem: No
- dMemRd_o=0; Read from D-Mem: No
- resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
- branch_s=0; Branch: No
- jump_s=0; Only for jalr, jal
- aluSel_o=various
- * opcode=35: stores
 - regWr_o=0; Write to register file: No
 - immSrc_o=3'b001; Instruction format: S-type
 - aluSrcB_o=1; Mux - Second register or immediate: immediate
 - aluSrcA_o=0; Mux - First register or PC: register
 - dMemWr_o=1; Write to D-Mem: Yes
 - dMemRd_o=0; Read from D-Mem: No
 - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
 - branch_s=0; Branch: No
 - jump_s=0; Only for jalr, jal
 - aluSel_o=5'b00000 (ADD)
- * opcode=51: arithmetic, logic and shifts
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'bxxx; Instruction format: R-type (don't care)
 - aluSrcB_o=0; Mux - Second register or immediate: register
 - aluSrcA_o=0; Mux - First register or PC: register
 - dMemWr_o=0; Write to D-Mem: No
 - dMemRd_o=0; Read from D-Mem: No
 - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
 - branch_s=0; Branch: No
 - jump_s=0; Only for jalr, jal
 - aluSel_o=various
- * opcode=55: lui
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'b100; Instruction format: U-type
 - aluSrcB_o=1; Mux - Second register or immediate: immediate

- aluSrcA_o=0; Mux - First register or PC: register
- dMemWr_o=0; Write to D-Mem: No
- dMemRd_o=0; Read from D-Mem: No
- resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
- branch_s=0; Branch: No
- jump_s=0; Only for jalr, jal
- aluSel_o=5'b00000 (ADD)
- * opcode=59: arithmetic, logic and shifts (64 bit)
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'bxxx; Instruction format: R-type (don't care)
 - aluSrcB_o=0; Mux - Second register or immediate: register
 - aluSrcA_o=0; Mux - First register or PC: register
 - dMemWr_o=0; Write to D-Mem: No
 - dMemRd_o=0; Read from D-Mem: No
 - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
 - branch_s=0; Branch: No
 - jump_s=0; Only for jalr, jal
 - aluSel_o=various
- * opcode=99: branches
 - regWr_o=0; Write to register file: No
 - immSrc_o=3'bxxx; Instruction format: B-type
 - aluSrcB_o=0; Mux - Second register or immediate: register
 - aluSrcA_o=0; Mux - First register or PC: register
 - dMemWr_o=0; Write to D-Mem: No
 - dMemRd_o=0; Read from D-Mem: No
 - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
 - branch_s=1; Branch: Yes
 - jump_s=0; Only for jalr, jal
 - aluSel_o=various
- * opcode=103: jalr
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'b000; Instruction format: I-type
 - aluSrcB_o=1; Mux - Second register or immediate: immediate

- aluSrcA_o=0; Mux - First register or PC: register
- dMemWr_o=0; Write to D-Mem: No
- dMemRd_o=0; Read from D-Mem: No
- resultSrc_o=2'b10; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: PC+4 to RegFile
- branch_s=0; Branch: No
- jump_s=1; Only for jalr, jal
- aluSel_o=5'b00000 (ADD)
- * opcode=111: jal
 - regWr_o=1; Write to register file: Yes
 - immSrc_o=3'b011; Instruction format: J-type
 - aluSrcB_o=0; Mux - Second register or immediate: register
 - aluSrcA_o=0; Mux - First register or PC: register
 - dMemWr_o=0; Write to D-Mem: No
 - dMemRd_o=0; Read from D-Mem: No
 - resultSrc_o=2'b10; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: PC+4 to RegFile
 - branch_s=0; Branch: No
 - jump_s=1; Only for jalr, jal
 - aluSel_o=5'b00000 (ADD)
- Signal: func3_i
 - Width: 3 bit, unsigned
 - Source: Instruction from I-Mem.
 - Destination: Case statement generating controls_s and aluSel_o
 - Explanation:
 - * opcode=19
 - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b00001 (SUBI) else aluSel_o=5'b00000 (ADDI)
 - func3_i=1: aluSel_o=5'b01111 (SLLI)
 - func3_i=2: aluSel_o=5'b00101 (SLTI)
 - func3_i=3: aluSel_o=5'b00110 (SLTIU)
 - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01110 (SRLI) else aluSel_o=5'b01101 (SRAI)
 - func3_i=4: aluSel_o=5'b00100 (XORI)

- func3_i=6: aluSel_o=5'b00011 (ORI)
- func3_i=7: aluSel_o=5'b00010 (ANDI)
- * opcode=27
 - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b01011 (SUBIW) else aluSel_o=5'b01100 (ADDIW)
 - func3_i=1: aluSel_o=5'b01010 (SLLIW)
 - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01001 (SRLIW) else aluSel_o=5'b01000 (SRAIW)
- * opcode=51
 - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b00001 (SUB) else aluSel_o=5'b00000 (ADD)
 - func3_i=1: aluSel_o=5'b01111 (SLL)
 - func3_i=2: aluSel_o=5'b00101 (SLT)
 - func3_i=3: aluSel_o=5'b00110 (SLTU)
 - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01110 (SRL) else aluSel_o=5'b01101 (SRA)
 - func3_i=4: aluSel_o=5'b00100 (XOR)
 - func3_i=6: aluSel_o=5'b00011 (OR)
 - func3_i=7: aluSel_o=5'b00010 (AND)
- * opcode=59
 - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b01011 (SUBW) else aluSel_o=5'b01100 (ADDW)
 - func3_i=1: aluSel_o=5'b01010 (SLLW)
 - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01001 (SRLW) else aluSel_o=5'b01000 (SRAW)
- * opcode=99
 - func3_i=0: aluSel_o=5'b10001 (BEQ, SUB)
 - func3_i=1: aluSel_o=5'b10011 (BNE, SUB)
 - func3_i=4: aluSel_o=5'b10000 (BLT, <)
 - func3_i=5: aluSel_o=5'b10010 (BGE, >=)
 - func3_i=6: aluSel_o=5'b10100 (BLTU, <)
 - func3_i=7: aluSel_o=5'b10101 (BGEU, >=)
- Signal: func7b5_i
 - Width: 1 bit, unsigned
 - Source: Instruction from I-Mem.

- Destination: Case statement generating controls_s and aluSel_o, rType-Subtract_s
- Explanation: Differentiates between ADD and SUB, shift variants
- Signal: zero_i
 - Width: 1 bit, unsigned
 - Source: ALU
 - Destination: zero_s
 - Explanation: Indicates that a branch should be taken.
- Signal: zero_s
 - Width: 1 bit, unsigned
 - Source: zero_i
 - Destination: PCSrc_o
 - Explanation: Indicates that a branch should be taken.

$$PCSrc_o = (branch_s \& zero_s) | jump_s$$

- Signal: branch_s
 - Width: 1 bit, unsigned
 - Source: generated from different instructions, see opcode_i
 - Destination: PCSrc_o
 - Explanation: Indicates that a branch should be taken.

$$PCSrc_o = (branch_s \& zero_s) | jump_s$$

- Signal: jump_s
 - Width: 1 bit, unsigned
 - Source: generated from different instructions, see opcode_i
 - Destination: PCSrc_o
 - Explanation: Indicates that a branch should be taken.

$$PCSrc_o = (branch_s \& zero_s) | jump_s$$

- Signal: rTypeSubtract_s

- Width: 1 bit, unsigned
- Source:

$$rTypeSubtract_s = func7b5_i \& opcode_i[5]$$

- Destination: Case statement generating controls_s and aluSel_o
- Explanation: Short form for distinguishing between ADD AND SUB.

- Signal: resultSrc_o

- Width: 2 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: Mux behind D-Mem
- Explanation:

00	for ALU-result to RegFile
01	for D-Mem-data to regFile
10	for PC+4 to RegFile

- Signal: dMemWr_o

- Width: 1 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: D-Mem
- Explanation: Write-enable for D-Mem

- Signal: dMemRd_o

- Width: 1 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: D-Mem
- Explanation: Read-enable for D-Mem; check if really needed

- Signal: PCSrc_o

- Width: 1 bit, unsigned
- Source:

$$PCSrc_o = (branch_s \& zero_s) | jump_s$$
- Destination: PC

- Explanation: Select input for the PC MUX.

0 $\Rightarrow$ PC+4 (PCp4) to PC

1 $\Rightarrow$ branch target address (PCbr) to PC

- Signal: aluSrcB_o

- Width: 1 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: Select input for the MUX between RegFile and ALU
- Explanation:

0 $\Rightarrow$ 2nd RegFile output to 2nd source input of ALU

1 $\Rightarrow$ immediate value to 2nd source input of ALU

- Signal: aluSrcA_o

- Width: 1 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: Select input for the MUX between RegFile and ALU
- Explanation:

0 $\Rightarrow$ 1st RegFile output to 1st source input of ALU

1 $\Rightarrow$ PC to 1st source input of ALU

- Signal: regWr_o

- Width: 1 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: Write enable for the RegFile
- Explanation: Write enable for the RegFile

- Signal: jump_o

- Width: 1 bit, unsigned
- Source:

$$jump_o = jump_s \& (\sim immSrc_o[0]) \& (\sim immSrc_o[1]) \& (\sim immSrc_o[2])$$

- Destination: Select input for the MUX in front of the branch target adder

- Explanation:

0 $\Rightarrow$ PC to adder input A
1 $\Rightarrow$ regA to adder input A

- Signal: immSrc_o

- Width: 3 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: Immediate generator control
- Explanation:

000 $\Rightarrow$ I-type
001 $\Rightarrow$ S-type
010 $\Rightarrow$ B-type
011 $\Rightarrow$ J-type
100 $\Rightarrow$ U-type

- Signal: aluSel_o

- Width: 5 bit, unsigned
- Source: generated from different instructions, see opcode_i
- Destination: ALU control

- Explanation:

00000 ⇒ ADD, ADDI, (JAL, JALR, AUIPC, LUI, stores, loads)
 00001 ⇒ SUB, SUBI
 00010 ⇒ AND, ANDI
 00011 ⇒ OR, ORI
 00100 ⇒ XOR, XORI
 00101 ⇒ SLT, SLTI
 00110 ⇒ SLTU, SLTIU
 00111 ⇒ tbd
 01000 ⇒ SRAW, SRAIW
 01001 ⇒ SRLW, SRLIW
 01010 ⇒ SLLW, SLLIW
 01011 ⇒ SUBW, SUBIW
 01100 ⇒ ADDW, ADDIW
 01101 ⇒ SRA, SRAI
 01110 ⇒ SRL, SRLI
 01111 ⇒ SLL, SLLI
 10000 ⇒ BLT
 10001 ⇒ BEQ
 10010 ⇒ BGE
 10011 ⇒ BNE
 10100 ⇒ BLTU
 10101 ⇒ BGEU

- Signal: branch_s_o

- Width: 1 bit, unsigned
- Source: branch_s
- Destination: Branch control for a pipelined solution
- Explanation: pipeline solution only

- Signal: jump_s_o

- Width: 1 bit, unsigned
- Source: jump_s
- Destination: Branch control for a pipelined solution
- Explanation: pipeline solution only

5.2.4 Service Requests of the Instruction Decoder

None.

5.2.5 The InstrDec Peripheral Kernel

The Instruction Decoder decodes all RV64I instructions and delivers the control signals to all design elements in the CPU.

Table 5.15 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Legend:

- Ins: Instruction short
- op: opcode_i
- f3: func3_i
- 7: func7b5_i
- z: zero_i
- S: resultSrc_o
- W: dMemWr_o
- R: dMemRd_o
- P: PCSrc_o
- B: aluSrcB_o
- A: aluSrcA_o
- r: regWr_o
- j: jump_o
- im: immSrc_o
- aluSel: aluSel_o
- b: branch_s_o
- js: jump_s_o

Table 5.15: RV32I/RV64I Base Integer Instruction Set [2], [3], [1]

Ins	op	f3	7	z	S	W	R	P	B	A	r	j	im	aluSel	b	js
lb	0000011	000	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lh	0000011	001	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lw	0000011	010	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lbu	0000011	100	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lhu	0000011	101	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
ld	0000011	011	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
ldu	0000011	110	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
addi	0010011	000	-	-	00	0	0	0	1	0	1	0	000	00000	0	0
slli	0010011	001	0	-	00	0	0	0	1	0	1	0	000	01111	0	0
slti	0010011	010	-	-	00	0	0	0	1	0	1	0	000	00101	0	0
sltiu	0010011	011	-	-	00	0	0	0	1	0	1	0	000	00110	0	0
xori	0010011	100	-	-	00	0	0	0	1	0	1	0	000	00100	0	0
srli	0010011	101	0	-	00	0	0	0	1	0	1	0	000	01110	0	0
srai	0010011	101	1	-	00	0	0	0	1	0	1	0	000	01101	0	0
ori	0010011	110	-	-	00	0	0	0	1	0	1	0	000	00011	0	0
andi	0010011	111	-	-	00	0	0	0	1	0	1	0	000	00010	0	0
auipc	0010111	-	-	-	00	0	0	0	1	1	1	0	100	00000	0	0
addiw	0011011	000	-	-	00	0	0	0	1	0	1	0	000	01100	0	0
slliw	0011011	001	0	-	00	0	0	0	1	0	1	0	000	01010	0	0
srliw	0011011	101	0	-	00	0	0	0	1	0	1	0	000	01001	0	0
sraiw	0011011	101	1	-	00	0	0	0	1	0	1	0	000	01000	0	0
sb	0100011	000	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
sh	0100011	001	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
sw	0100011	010	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
sd	0100011	011	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
add	0110011	000	0	-	00	0	0	0	0	0	1	0	000	00000	0	0
sub	0110011	000	1	-	00	0	0	0	0	0	1	0	000	00001	0	0
sll	0110011	001	0	-	00	0	0	0	0	0	1	0	000	01111	0	0
slt	0110011	010	0	-	00	0	0	0	0	0	1	0	000	00101	0	0
sltu	0110011	011	0	-	00	0	0	0	0	0	1	0	000	00110	0	0
xor	0110011	100	0	-	00	0	0	0	0	0	1	0	000	00100	0	0
srl	0110011	101	0	-	00	0	0	0	0	0	1	0	000	01110	0	0
sra	0110011	101	1	-	00	0	0	0	0	0	1	0	000	01101	0	0
or	0110011	110	0	-	00	0	0	0	0	0	1	0	000	00011	0	0
and	0110011	111	0	-	00	0	0	0	0	0	1	0	000	00010	0	0

... next page

Ins	op	f3	7	z	S	W	R	P	B	A	r	j	i m	aluSel	b	js
lui	0110111	-	-	-	00	0	0	0	1	0	1	0	100	00000	0	0
addw	0111011	000	0	-	00	0	0	0	0	0	1	0	000	01100	0	0
subw	0111011	000	1	-	00	0	0	0	0	0	1	0	000	01011	0	0
sllw	0111011	001	0	-	00	0	0	0	0	0	1	0	000	01010	0	0
srlw	0111011	101	0	-	00	0	0	0	0	0	1	0	000	01001	0	0
sraw	0111011	101	1	-	00	0	0	0	0	0	1	0	000	01000	0	0
beq	1100011	000	-	0/1	00	0	0	0/1	0	0	0	0	010	10001	1	0
bne	1100011	001	-	0/1	00	0	0	0/1	0	0	0	0	010	10011	1	0
blt	1100011	100	-	0/1	00	0	0	0/1	0	0	0	0	010	10000	1	0
bge	1100011	101	-	0/1	00	0	0	0/1	0	0	0	0	010	10010	1	0
bltu	1100011	110	-	0/1	00	0	0	0/1	0	0	0	0	010	10100	1	0
bgeu	1100011	111	-	0/1	00	0	0	0/1	0	0	0	0	010	10101	1	0
jalr	1100111	000	-	-	10	0	0	1	1	0	1	0	000	00000	0	1
jal	1101111	-	-	-	10	0	0	1	0	0	1	0	011	00000	0	1

EoT

5.2.6 Registers

None.

Chapter 6

Peripherals

6.1 GPIO

6.1.1 System Integration of GPIO0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.1: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.2: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)
2025-11-19	Andreas Siggelkow	Changed the BPI; removed the registers from the kernel and added them to GPIOTop. Added an interrupt. Removed the address output. changed the GPIO output to inout.

Functional Configuration of GPIO0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a GPOI peripheral are described in its functional description in Section 6.1.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for GPIO0 in Table 6.3.

Table 6.3: GPIO0 Feature Set

	Generic GPIO Feature Set	Details
yes	Seperate Kernel Clock	125 MHz
no	FIFO Data Buffering	not yet

HW Interfaces

The following table 6.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 6.4: HW Interfaces of Module GPIO0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	gpio_irq_o	gpio_irq_s
DMA	None	-
External Signals	gpio_io cs_o	gpio_io asGpioCs_s

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.5: Register List of Module GPIO0

Register Name(s) in Chip	Name used in module register description	Comment Comment
GPIO0_ID	id_reg_s	ID of GPIO0 peripheral
GPIO0_DIR	dir_reg_s	Direction of the GPIO: • dir_reg_s[i] = 0 -> output, • dir_reg_s[i] = 1 -> input
GPIO0_DATA	data_reg_s	Data driven to the GPIO or received from the GPIO.
GPIO0_IRQSS	irqss_reg_s	GPIO Interrupt Request Source Status Register
GPIO0_IRQSC	irqsc_reg_s	GPIO Interrupt Request Source Clear Register
GPIO0_IRQSM	irqsm_reg_s	GPIO Interrupt Request Source Mask Register
GPIO0_ISR	isr_reg_s	GPIO Interrupt Set Register
GPIO0_RIS	ris_reg_s	GPIO Raw Interrupt Status Register
GPIO0_IMSC	imsc_reg_s	GPIO Interrupt Mask Control Register
GPIO0_MIS	mis_reg_s	GPIO Masked Interrupt Status Register

Related Sections of this Spec

- GPIO functional description starting with Section 6.1.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
 - Slave WB (Wishbone) BPI Section “Architecture Concepts”
 - Peripheral ID Registers Section “Architecture Concepts”
 - Peripheral Clocking Concept Section “Architecture Concepts”
 - Clock Control Registers Section “Architecture Concepts”

- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.1.2 History of GPIO

Table 6.6: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
6.1	6.1 / 2025-02-13	First draft (AS)
6.1	6.1 / 2025-11-19	Changed the BPI; removed the registers from the kernel and added them to GPIOTop. Added an interrupt. Removed the address output. changed the GPIO output to inout. (AS)

6.1.3 Functional Overview

The GPIO peripheral allows to drive 8 bit data from the Wishbone data bus to GPIO pins gpio_io (7:0) (input and output). The direction must be programmed

with the **GPIO0_DIR** register. A chip select (cs) is used to activate the GPIO output (dummy citation [4]).

General Features:

- Chip select
- FIFO data buffering (future)
- I/O
- Interrupt

Synchronous Mode:

- 8 data bits

Asynchronous Mode:

- -

6.1.4 Structural Overview

The GPIO is a Wishbone slave (asSlaveBPI) peripheral. This peripheral architecture aims for high bus performance by dividing peripherals into two clock domains. Figure 6.1 shows a simplified block diagram of the GPIO asSlaveBPI peripheral.

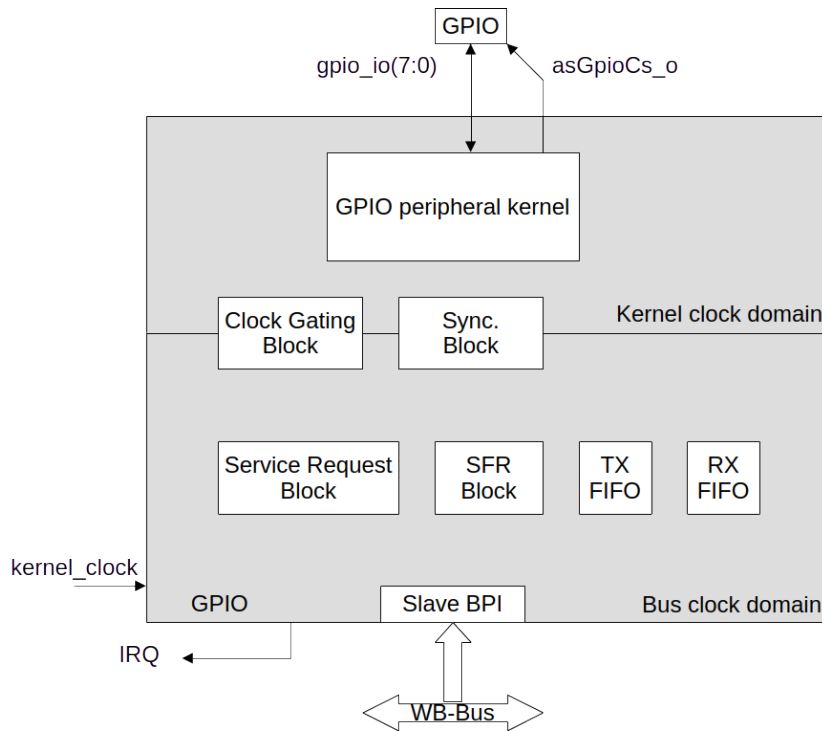


Figure 6.1: GPIO asSlaveBPI Peripheral

GPIO Peripheral Kernel: The GPIO kernel provides the specific functionality of the GPIO interface, that is described in Section 6.1.6. The peripheral kernel has its own kernel clock domain.

Bus Peripheral Interface BPI: The BPI is responsible for the interconnection of the GPIO registers with an on-chip bus. Also the peripheral clock gating is controlled by the BPI.

asSlaveBPI Block: The asSlaveBPI block lies within the bus clock domain and contains the following subblocks:

Special Function Register Block The Special Function Register Block contains all FIFO specific registers and all special function registers (SFR) of the peripheral kernel that are accessible by the SW via the system bus.

Service Request Block SRB The SRB is used to prepare the interrupt and data transfer requests for the Interrupt Control Unit (ICU) and the DMA Controller (DMAC), respectively (no DMA planned!).

TX FIFO The TX FIFO is used to buffer the transmission data from the bus, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the system bus.

RX FIFO The RX FIFO is used to buffer the received data from the kernel, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the bus system.

Clock Gating Block: The Clock Gating Block is used to derive the necessary clocks for the several blocks within the peripheral from the clocks provided by the system.

Synchronization Block: The Synchronization Block is used to synchronize the signals between the bus clock and kernel clock domains.

I/O of the GPIO Kernel

Table 6.7 shows the input and outputs of the GPIO kernel.

Table 6.7: GPIO Kernel I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
direction_i(7:0)	in	8	From GPIO-BPI (GPIO0_DIR register). Defines the direction of each GPIO. If the bit is 0, the GPIO is an output, if the bit is 1, the GPIO is an input.
addr_i(3:0)	in	4	From GPIO-BPI. Address for storing cs_s.
data_i(7:0)	in	8	From GPIO-BPI (GPIO0_DATA register). GPIO data in output direction. The elements of the register will be forwarded to a tristate buffer which is active when direction is output and 'Z' when direction is input.
data_o(7:0)	out	8	From pad to GPIO-BPI (GPIO0_DATA register). GPIO data in input direction. If the direction is input, the signals from the pads will go through a tristate buffer and will be forwarded to the register. Else, a '0' will be driven.
irq_o(7:0)	out	8	From kernel to GPIO-BPI. A change of an input will be detected and send to the IRQ register. Every possible GPIO input will be routed to an edge detector (both, rising and falling). Those detected edges will be forwarded to the IRQ structure in the BPI.
en_i	in	1	From GPIO-BPI. Enables a write to a GPIO block. It stores the chip select cs_o.
gpio_io	bidirectional	8	To inout pins. GPIO inout.
cs_o	out	1	To output pins. Chip select for outside GPIO block. Stored in an internal register (so, synchronized to the data).

I/O of the GPIO BPI

Table 6.8 shows the input and outputs of the GPIO BPI.

Table 6.8: GPIO BPI I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
addr_o(3:0)	out	4	To GPIO kernel. Address for external GPIO devices. addr_o = addr_i.
dat_from_core_i(7:0)	in	8	From GPIO kernel.
dat_to_core_o(7:0)	out	8	To GPIO kernel. GPIO outputs for one block. dat_to_core_o = dat_i
wr_o	out	1	To GPIO kernel. Enable for GPIO data and address, and chip select for outside GPIO blocks. wr_o = we_i AND stb_i.
addr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
dat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
dat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
we_i	in	1	From WB-Bus. Peripherals write enable.
sel_i(7:0)	in	8	From WB-Bus. Peripherals byte select. Not used here.
stb_i	in	1	From WB-Bus. Valid bus cycle. Combined (AND) with we_i for wr_o.
ack_o	out	1	To WB-Bus. Error free bus transaction. Here, ack_o = stb_i.
cyc_i	in	1	From WB-Bus. High for complete bus cycle. Not used here.

Internals of the GPIO BPI

Table 6.9 shows the internal signals and processes of the GPIO BPI.

Table 6.9: GPIO BPI Internals

Object	Type	Width	Explanation
any	logic	x	See BPI spec.

I/O of the GPIO Top

Table 6.10 shows the input and outputs of the GPIO Top.

Table 6.10: GPIO Top I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
wbdAddr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
wbdDat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
wbdDat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
wbdWe_i	in	1	From WB-Bus. Peripherals write enable.
wbdSel_i(7:0)	in	8	From WB-Bus. Peripherals byte select.
wbdStb_i	in	1	From WB-Bus. Valid bus cycle.
wbdAck_o	out	1	To WB-Bus. Error free bus transaction.
wbdCyc_i	in	1	From WB-Bus. High for complete bus cycle.
gpio_irq_o	out	1	GPIO interrupt.
gpio_o(7:0)	out	8	To chip pins. GPIO outputs for one block.
cs_o	out	1	To chip pins. Chip select for outside GPIO blocks.

Table 6.11 shows the internals of the GPIO Top.

Table 6.11: GPIO Top Signals

Signal	Width	Explanation
irq_comb_s	1	An ORed signal of all interrupts coming from the GPIO kernel.
irqsc_comb_s	1	An ORed signal of all interrupt clear bits from register IRQSC.
irqss_reg_s	64	GPIO Interrupt Request Source Status Register GPIO0_IRQSS . If irq_comb_s = 1, the interrupts from the kernel will be written. Else, if irqsc_comb_s = 1, irqss_reg_s will be cleared and irqsc_reg_s will be cleared.
irqsc_reg_s	64	GPIO Interrupt Request Source Clear Register GPIO0_IRQSC . One interrupt request source clear bit is provided for each interrupt request source: 0 - No change, 1 - Clear Interrupt source.
irqsm_reg_s	64	GPIO Interrupt Request Source Mask Register GPIO0_IRQSM . One interrupt request source mask bit is provided for each interrupt request source: 0 - Interrupt Request Source Disabled, 1 - Interrupt Request Source Enabled.
irqsm_comb_s	1	An ORed signal of all masked single interrupts coming from the GPIO kernel.
ris_reg_s	64	GPIO Raw Interrupt Status Register GPIO0_RIS . One interrupt status bit is provided for each service request: 0 - No Interrupt, 1 - Interrupt Pending.
isr_reg_s	64	GPIO Interrupt Set Register GPIO0_ISR . One interrupt set bit is provided for each service request. The interrupt status bits in the GPIO0_RIS and the MIS (if IMSC = 1) registers are set by writing a '1' to the desired positions. 0 - No change, 1 - Set Interrupt. Reading the register returns 0.
imsc_reg_s	64	GPIO Interrupt Mask Control Register GPIO0_IMSC . One interrupt mask bit is provided for each service request: 0 - Interrupt Disabled, 1 - Interrupt Enabled.
mis_reg_s	64	GPIO Masked Interrupt Status Register GPIO0_MIS . One interrupt status bit is provided for each service request: 0 - No Interrupt, 1 - Interrupt Pending.
id_reg_s	64	GPIO ID Register GPIO0_ID .
dir_reg_s	64	GPIO Direction Register GPIO0_DIR .
data_reg_s	64	GPIO Data Register GPIO0_DATA . If the direction is input, the values from the pad will be written to the register. If the direction is output, the values from the bus will be written to the register.

6.1.5 Service Requests of the GPIO

Table 6.12 shows the service requests of the GPIO.

Table 6.12: GPIO Service Requests

Service Request	IRQ	Explanation
gpio_io (0)	GPIO_INT	The input on gpio_io (0) changed.
gpio_io (1)	GPIO_INT	The input on gpio_io (1) changed.
gpio_io (2)	GPIO_INT	The input on gpio_io (2) changed.
gpio_io (3)	GPIO_INT	The input on gpio_io (3) changed.
gpio_io (4)	GPIO_INT	The input on gpio_io (4) changed.
gpio_io (5)	GPIO_INT	The input on gpio_io (5) changed.
gpio_io (6)	GPIO_INT	The input on gpio_io (6) changed.
gpio_io (7)	GPIO_INT	The input on gpio_io (7) changed.

6.1.6 The GPIO Peripheral Kernel

As common for asSlaveBPI peripherals there are also two main states of the GPIO's kernel.

Before it can actually start operation, it has to be configured. This configuration state is left by setting bit RUN in register RUN_CTRL. Thereby the configuration registers are locked for write accesses, the functional blocks of peripheral kernel and asSlaveBPI block are reset (e.g. the FIFOs are flushed) and working state is entered. The different functional blocks of the GPIO peripheral kernel start operating as described in the following.

Figure 6.2 shows the principal functionality of the GPIO kernel. The shown registers are part of the SFR block of the peripheral. When there is a '1' written to the direction register, the pad acts as an input and the captured value will be written to the data register. When there is a '0' written to the direction register, the pad acts as an output and the value from the data register will be written to the pad. The data register is rw from the bus side.

In case of an input setting, a signal change will be detected and written to the IRQ register inside the SRB.

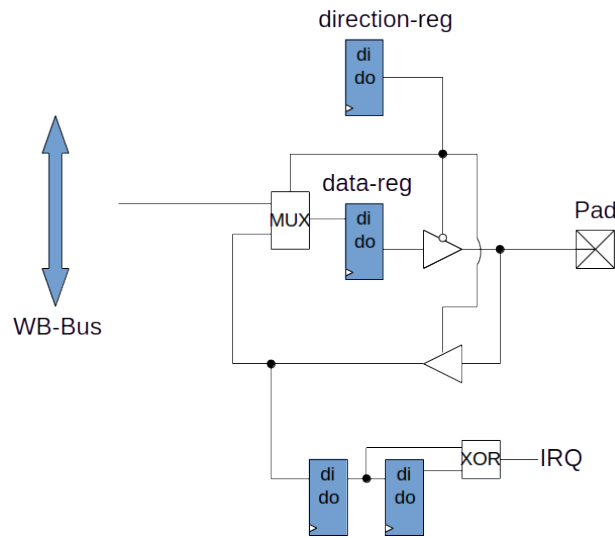


Figure 6.2: GPIO Kernel

6.1.7 Registers

In the following the registers of a GPIO peripheral are listed and described. Some registers' layout or availability depends on the generic feature set implemented for that GPIO's instantiation. Such registers and concerned bit fields are denoted by the use of italic style.

GPIO Register Overview

Table 6.13: GPIO Register List

Register Group	Register Name	Register Symbol
System Registers	(Clock Control Register) Identification Register	(CLC) ID
Special Function Registers	(Run Control Register) Direction Register Data Register	(RUN_CTRL) GPIO0_DIR GPIO0_DATA
Interrupt Registers	Interrupt Mask Control Register Raw Interrupt Status Register Masked Interrupt Status Register Interrupt Set Register Interrupt Source Status Register Interrupt Source Mask Register Interrupt Source Clear Register	GPIO0_IMSC GPIO0_RIS GPIO0_MIS GPIO0_ISR GPIO0_IRQSS GPIO0_IRQSM GPIO0_IRQSC

Read-Write Features

- w: writable bit, by SW
- r: readable bit, by SW
- h: bit will be set by HW only

System Registers

GPIO_ID -: Identification register of this peripheral. A write will have no effect, a read will return the fixed ID. The ID will be defined on chip level.

ID register (in BPI)

[Reset value: 000100000000C001_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
MOD_NUMBER															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
1	1	0	0	0	0	0	0	REV_NUMBER							
r	r	r	r	r	r	r	r	r							

Field	Bits	Type	Description
MOD_NUMBER	63:48	r	Module Identification Number.
-	47:16	r	Reserved
-	15:8	r	The value C0 indicates the peripheral as 64-bit module.
REV_NUMBER	7:0	r	Module Revision Number: Bits 7-0 are used for module revision numbering.

Special Function Registers

Interrupt Registers

6.2 Instruction Memory (I-Mem)

6.2.1 System Integration of I-Mem0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.14: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	November 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.15: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-11-27	Andreas Siggelkow	First draft (AS)

Functional Configuration of I-Mem0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a **I-Mem** peripheral are described in its functional description in Section 6.2.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for **I-Mem0** in Table 6.16.

Table 6.16: **I-Mem0** Feature Set

	Generic I-Mem Feature Set	Details
yes	Seperate Kernel Clock	125 MHz
no	FIFO Data Buffering	no

HW Interfaces

The following table 6.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 6.17: HW Interfaces of Module **I-Mem0**

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	None	-
DMA	None	-
External Signals	addr_i data_o data_i wr_i	iBusAddr2_s iBusDataRd_s iBusDataWr_s iMemWr_s

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.18: Register List of Module **I-Mem0**

Register Name(s) in Chip	Name used in module register description	Comment Comment
None	-	-

Related Sections of this Spec

- **I-Mem** functional description starting with Section 6.2.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
 - Slave WB (Wishbone) BPI Section “Architecture Concepts”
 - Peripheral ID Registers Section “Architecture Concepts”
 - Peripheral Clocking Concept Section “Architecture Concepts”
 - Clock Control Registers Section “Architecture Concepts”

- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.2.2 History of I-Mem

Table 6.19: History

History			
Target Spec.		Current version : 1.0, 2025-11-27 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	(in ver- sion)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-11-27			
6.2		6.2 / 2025-11-27	First draft (AS)

6.2.3 Functional Overview

The **I-Mem** peripheral is the internal instruction memory of the **RWU-RV64I**.

General Features:

- 32 kByte ROM (8192 words/instructions)
- Linear addressable
- 32 bit data width
- Wishbone bus slave

Synchronous Mode:

- 32 data bits (write; program download via JTAG)

Asynchronous Mode:

- 32 data bits (read; normal operation)

6.2.4 Structural Overview

The **I-Mem** is a Wishbone slave (asSlaveBPI) peripheral. It has one write port (for JTAG download) and one read port (for normal operation). The address is shared.

I/O of the I-Mem Kernel

Table 6.20 shows the input and outputs of the **I-Mem** kernel.

Table 6.20: **I-Mem** Kernel I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
addr_i	in	15	From program counter (PC). There is no byte access, so the lower two address bits will not be needed. In case of a program download, this is a part of the JTAG scan-chain (outside the block).
data_o	out	32	Instruction. Goes to the instruction decoder.
data_i	in	32	Data input port. From JTAG scan-chain (JTAG data register). This port is used for a program download.
wr_i	in	1	Write enable for data download. From JTAG scan-chain (JTAG data register). This port is used for a program download.

6.2.5 Service Requests of the I-Mem

None.

6.2.6 The I-Mem Peripheral Kernel

The memory is implemented as a synchronous write, asynchronous read memory. The write will be done on the rising edge of TCK and a write enable from a JTAG scan-chain.

6.2.7 Registers

None.

6.3 QSPI

6.3.1 System Integration of QSPI0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.21: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2026

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.22: History

Date	Name	Content Changes
Version of this document: V0.1		
2026-02-16	Andreas Siggelkow	First draft (AS, Chat-GPT)

Functional Configuration of QSPI0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a QSPI peripheral are described in its functional description in Section 6.3.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for QSPI0 in Table 6.23.

Table 6.23: QSPI0 Feature Set

	Generic QSPI Feature Set	Details
yes	Separate Kernel Clock	125 MHz (TBD)
yes	FIFO Data Buffering	RX and TX, each 16 entries $\times$ 64 bit = 128 bytes
yes	Programmable Baud Rate	$f_{SCK} = f_{kernel} / (2 \times (CLKDIV + 1))$, CLK-DIV $\in [0 \dots 255]$
yes	Single SPI Mode	1-1-1, cmd/adr/data on one wire
yes	Dual SPI Mode	1-1-2, data on two wires (DUAL=1, QUAD=0)
yes	Quad SPI Mode	1-1-4 and 1-4-4, data on four wires (QUAD=1)
yes	Configurable SPI Mode	CPOL and CPHA selectable (Mode 0 and Mode 3)
yes	3-/4-Byte Address	Selectable via ADDR_LEN bit
yes	Execute-In-Place (XIP)	Auto-read on bus access, fixed opcode from CMD register
yes	Double Data Rate (DDR)	DDR bit in CTRL register
yes	Interrupt Generation	7 interrupt sources, maskable via IMSC
yes	FIFO Flush	TX_FLUSH and RX_FLUSH bits in CTRL register
yes	Configurable Timeout	Timeout counter, threshold set via TIMEOUT register

HW Interfaces

The following table 6.24 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may

be different). A full list is part of the design spec.

Table 6.24: HW Interfaces of Module QSPI0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	qspi_irq_o	qspi_irq_s
DMA	None	-
External Signals	sck_o cs_o data_io[3:0]	qspiSck_o qspiCs_o qspi_data_io [3:0]

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.25: Register List of Module QSPI0

offset	Register Name(s) in Chip	Name used in module register description	Comment
0x00	QSPI0_ID	id_reg_s	ID of QSPI0 peripheral
0x08	QSPI0_CTRL	ctrl_reg_s	QSPI Control register
0x10	QSPI0_CMD	cmd_reg_s	SPI command opcode
0x18	QSPI0_ADDR	addr_reg_s	QSPI target flash address
0x20	QSPI0_LEN	len_reg_s	QSPI transfer length in bytes
0x28	QSPI0_DUMMY	dummy_reg_s	QSPI dummy cycles
0x30	QSPI0_STATUS	status_reg_s	QSPI status flags
0x38	QSPI0_TXDATA	txdata_reg_s	QSPI TX FIFO write
0x40	QSPI0_RXDATA	rxdata_reg_s	QSPI RX FIFO read
0x48	QSPI0_FIFOSTAT	fifostat_reg_s	QSPI FIFO levels
0x50	QSPI0_IMSC	imsc_reg_s	QSPI Interrupt Mask Control Register
0x58	QSPI0_RIS	ris_reg_s	QSPI Raw Interrupt Status Register
0x60	QSPI0_ISR	isr_reg_s	QSPI Interrupt Set Register
0x68	QSPI0_ICR	icr_reg_s	QSPI Interrupt Clear Register
0x70	QSPI0_MIS	mis_reg_s	QSPI Masked Interrupt Status Register
0x78	QSPI0_CLKDIV	clkdiv_reg_s	QSPI Clock Divider register (Baud Rate)
0x80	QSPI0_TIMEOUT	timeout_reg_s	QSPI Timeout threshold register

Related Sections of this Spec

- QSPI functional description starting with Section 6.3.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”

- Slave WB (Wishbone) BPI Section “Architecture Concepts”
- Peripheral ID Registers Section “Architecture Concepts”
- Peripheral Clocking Concept Section “Architecture Concepts”
- Clock Control Registers Section “Architecture Concepts”
- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.3.2 History of QSPI

Table 6.26: History

History		
Target Spec.	Current version : 1.0, 2026-02-17 Author: A. Siggelkow Previous version : 0.0, 2020-10-12	
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2026-02-17		
6.1	6.1 / 2026-02-17	First draft (AS)

6.3.3 Functional Overview

The QSPI controller is a memory-mapped peripheral providing high-speed SPI and Quad-SPI access to external flash devices.

The controller is accessed via a 64-bit Wishbone slave interface (dummy citation [4]).

General Features:

- Single, Dual, and Quad SPI modes
- Configurable dummy cycles
- Programmable baud rate
- TX and RX FIFOs
- Interrupt generation
- Optional Execute-In-Place (XIP)

Synchronous Mode:

- 4 data bits

Asynchronous Mode:

- -

Reference Flash Devices

The QSPI controller is designed to be compatible with widely used NOR-Flash devices. Table 6.27 shows the relevant operating parameters for two representative devices. All register default values in this specification are set to match the Winbond W25Q128JV in standard (non-DDR) mode with CPOL=0, CPHA=0.

Table 6.27: Reference NOR-Flash Devices

Parameter	Winbond W25Q128JV	Micron MT25QL128	Comment
Capacity	128 Mbit (16 MByte)	128 Mbit (16 MByte)	
Supply Voltage	2.7 V...3.6 V	2.7 V...3.6 V	
SPI Modes supported	Mode 0 (CPOL=0, CPHA=0) and Mode 3 (CPOL=1, CPHA=1)	Mode 0 (CPOL=0, CPHA=0) and Mode 3 (CPOL=1, CPHA=1)	
Max. clock (STR)	133 MHz (Quad)	133 MHz (Quad)	
Default address width	3 Byte (24 bit)	3 Byte (24 bit)	4-Byte mode selectable
Quad enable	QE bit in Status Reg 2	via Enhanced Volatile Config Reg (0x61)	Must be set be- fore QUAD=1
Relevant Read Opcodes:			
Read (slow)	0x03 (1-1-1, 0 dummy)	0x03 (1-1-1, 0 dummy)	≤50 MHz
Fast Read	0x0B (1-1-1, 8 dummy)	0x0B (1-1-1, 8 dummy)	up to 133 MHz
Dual Output Read	0x3B (1-1-2, 8 dummy)	0x3B (1-1-2, 8 dummy)	DUAL=1
Quad Output Read	0x6B (1-1-4, 8 dummy)	0x6B (1-1-4, 8 dummy)	QUAD=1
Quad I/O Read	0xEB (1-4-4, 6+2 dummy)	0xEB (1-4-4, 10 dummy)	QUAD=1, fast
Relevant Write / Control Opcodes:			
Write Enable	0x06	0x06	Required before pro- gram/erase
Page Program	0x02 (1-1-1)	0x02 (1-1-1)	Up to 256 Byte
Sector Erase (4 KB)	0x20	0x20	
Block Erase (64 KB)	0xD8	0xD8	
Chip Erase	0xC7 / 0x60	0xC7 / 0x60	
JEDEC ID Read	0x9F	0x9F	Returns 3 Byte
JEDEC Manufacturer ID	0xEF (Win- bond)	0x20 (Micron)	
Typical Dummy Cycles by Mode and Clock:			
1-1-1 Fast Read	8 cycles	8 cycles	QSPI0_DUMMY = 0x08
1-1-4 Quad Output	8 cycles	8 cycles	QSPI0_DUMMY = 0x08
1-4-4 Quad I/O	6 mode + 2 dummy cycles	10 cycles	QSPI0_DUMMY = 0x08 / 0x0A

Note on Quad Enable: Both reference devices require a Quad Enable (QE) bit to be set in the flash device's non-volatile status register before QUAD mode can be used. This is a one-time (or volatile) configuration step performed by the SW driver via a standard Write Enable (0x06) followed by a Write Status Register command. The QSPI controller itself does not perform this step automatically.

6.3.4 Structural Overview

The QSPI is a Wishbone slave (asSlaveBPI) peripheral. This peripheral architecture aims for high bus performance by dividing peripherals into two clock domains. Figure 6.3 shows a simplified block diagram of the QSPI asSlaveBPI peripheral.

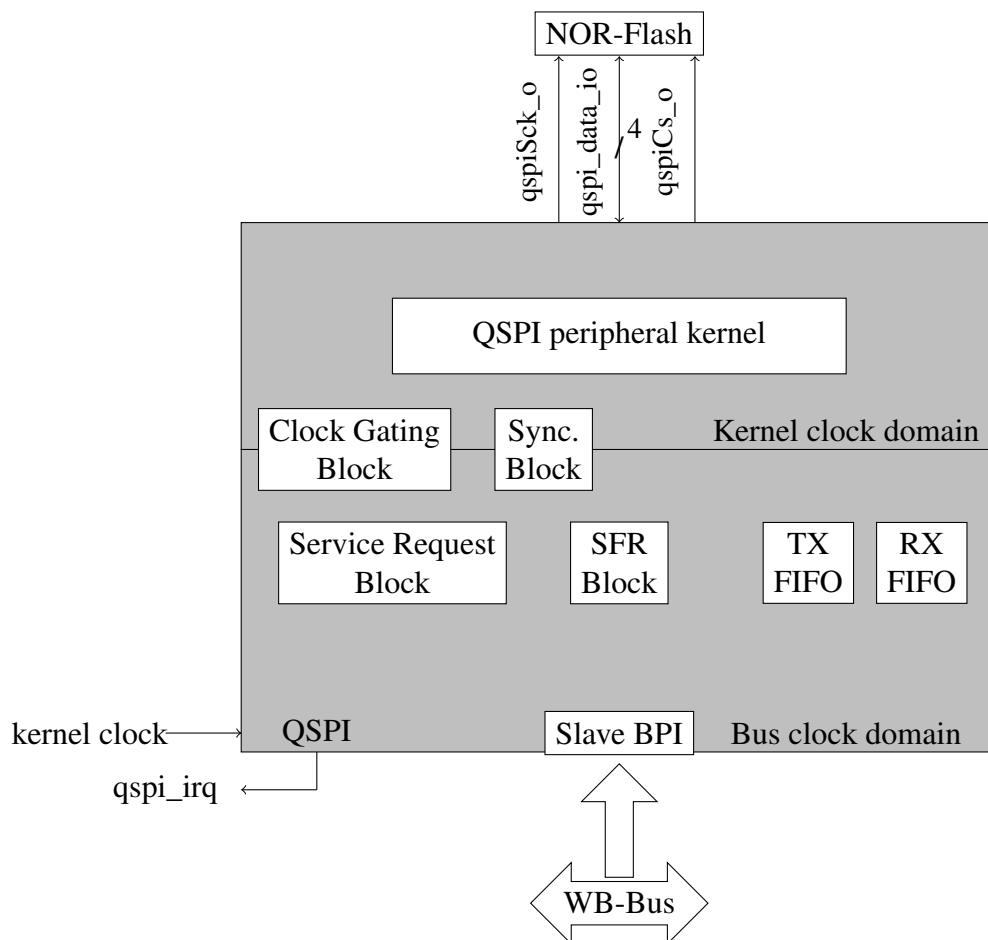


Figure 6.3: QSPI asSlaveBPI Peripheral

QSPI Peripheral Kernel: The QSPI kernel provides the specific functionality of the QSPI interface, that is described in Section 6.3.6. The peripheral kernel has its own kernel clock domain.

Bus Peripheral Interface BPI: The BPI is responsible for the interconnection of the QSPI registers with an on-chip bus. Also the peripheral clock gating is controlled by the BPI.

asSlaveBPI Block: The asSlaveBPI block lies within the bus clock domain and contains the following subblocks:

Special Function Register Block The Special Function Register Block contains all FIFO specific registers and all special function registers (SFR) of the peripheral kernel that are accessible by the SW via the system bus.

Service Request Block SRB The SRB is used to prepare the interrupt and data transfer requests for the Interrupt Control Unit (ICU) and the DMA Controller (DMAC), respectively (no DMA planned!).

TX FIFO The TX FIFO is used to buffer the transmission data from the bus, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the system bus.

RX FIFO The RX FIFO is used to buffer the received data from the kernel, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the bus system.

Clock Gating Block: The Clock Gating Block is used to derive the necessary clocks for the several blocks within the peripheral from the clocks provided by the system.

Synchronization Block: The Synchronization Block is used to synchronize the signals between the bus clock and kernel clock domains.

I/O of the QSPI Kernel

Table 6.28, 6.29, 6.30 shows the input and outputs of the QSPI kernel.

Table 6.28: QSPI Kernel I/O (1)

Pin	Direction	Width	Explanation
clk_i	in	1	Kernel clock
rst_i	in	1	Kernel reset. Active high.

Table 6.29: QSPI Kernel I/O (2)

Pin	Direction	Width	Explanation
Interface to BPI:			
ctrl_reg11_addr_i	in	1	From QSPI-BPI (QSPI0_CTRL register). Addr length (as member of an enum type)
ctrl_reg10_cpol_i	in	1	Clock polarity (enum type)
ctrl_reg9_cpha_i	in	1	Clock phase (enum type)
ctrl_reg8_dual_i	in	1	Dual data (enum type)
ctrl_reg7_csh_i	in	1	CS hold (enum type)
ctrl_reg6_xip_i	in	1	XIP (enum type)
ctrl_reg5_ddr_i	in	1	DDR (enum type)
ctrl_reg4_quad_i	in	1	Quad data (enum type)
ctrl_reg3_dual_i	in	1	Start (must be a pulse) (enum type)
cmd_reg_i(7:0)	in	8	From QSPI-BPI (QSPI0_CMD register). Defines the SPI command opcode.
addr_reg_i(31:0)	in	32	From QSPI-BPI (QSPI0_ADDR register). Defines the target flash address.
len_reg_i(15:0)	in	16	From QSPI-BPI (QSPI0_LEN register). Defines the transfer length in bytes.
dummy_reg_i(5:0)	in	6	From QSPI-BPI (QSPI0_DUMMY register). Defines the needed dummy cycles.
status_reg_o:	-	-	To QSPI0_STATUS reg. ...
stat_busy_o	out	1	FSM active (level).
stat_done_o	out	1	FSM in DONE state (pulse).
stat_error_o	out	1	Error latched (level, until ICR).
stat_timeout_o	out	1	Timeout (level, until ICR).
tx_empty_i	in	1	From TX-FIFO. Empty indication.
rx_full_i	in	1	From RX-FIFO. Full indication.
tx_rd_o	out	1	Read strobe to TX-FIFO.
rx_wr_o	out	1	Write strobe to RX-FIFO.
timeout_reg_i(31:0)	in	32	Timeout level.
tx_data_i(63:0)	in	64	From QSPI-BPI (TX-FIFO). Data to be send.
rx_data_o(63:0)	out	64	To QSPI-BPI (RX-FIFO). Data received.

Table 6.30: QSPI Kernel I/O (3)

Pin	Direction	Width	Explanation
Interface to PADs:			
qspi_data_io	bidirectional	4	To inout pins. QSPI data inout. Generated by the QSPI FSM.
qspiSck_o	out	1	To output pins. Clock for outside NOR-flash. Generated by the QSPI FSM.
qspiCs_o	out	1	To output pins. Chip select for outside NOR-flash. Generated by the QSPI FSM.

I/O of the QSPI BPI

Table 6.31 shows the input and outputs of the Qspi BPI.

Table 6.31: QSPI BPI I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
Interface to peripheral kernel:			
ctrl_reg_o(4:0)	out	5	To QSPI-kernel. Defines and starts the QSPI FSM.
cmd_reg_o(x:0)	out	x	To QSPI-kernel. Defines the SPI command opcode.
addr_reg_o(x:0)	out	x	To QSPI-kernel. Defines the target flash address.
len_reg_o(x:0)	out	x	To QSPI-kernel. Defines the transfer length in bytes.
dummy_reg_o(x:0)	out	x	To QSPI-kernel. Defines the needed dummy cycles.
status_reg_i(3:0)	in	4	From QSPI-kernel. Delivers BUSY, DONE, ERROR and TIMEOUT.
tx_data_o(63:0)	out	64	To QSPI-kernel (TX-FIFO). Data to be send.
rx_data_i(63:0)	in	64	From QSPI-kernel (RX-FIFO). Data received.
Interface to the bus:			
wbd_s_addr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
wbd_s_dat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
wbd_s_dat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
wbd_s_we_i	in	1	From WB-Bus. Peripherals write enable.
wbd_s_sel_i(7:0)	in	8	From WB-Bus. Peripherals byte select. Not used here.
wbd_s_stb_i	in	1	From WB-Bus. Valid bus cycle.
wbd_s_ack_o	out	1	To WB-Bus. Error free bus transaction.
wbd_s_cyc_i	in	1	From WB-Bus. High for complete bus cycle.
Other signals:			
qspi_irq_o	out	1	QSPI interrupt.

Internals of the QSPI BPI

Table 6.32 shows the internal signals and processes of the QSPI BPI.

Table 6.32: QSPI BPI Internals

Object	Type	Width	Explanation
any	logic	x	See BPI spec.

I/O of the QSPI Top

Table 6.33 shows the input and outputs of the QSPI Top.

Table 6.33: QSPI Top I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
wbd_s_addr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
wbd_s_dat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
wbd_s_dat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
wbd_s_we_i	in	1	From WB-Bus. Peripherals write enable.
wbd_s_sel_i(7:0)	in	8	From WB-Bus. Peripherals byte select.
wbd_s_stb_i	in	1	From WB-Bus. Valid bus cycle.
wbd_s_ack_o	out	1	To WB-Bus. Error free bus transaction.
wbd_s_cyc_i	in	1	From WB-Bus. High for complete bus cycle.
qspi_irq_o	out	1	QSPI interrupt.
qspi_data_io(3:0)	inout	4	To chip pins. QSPI data for NOR-flash.
qspiCs_o	out	1	To chip pins. Chip select for NOR-flash.
qspiSck_o	out	1	To chip pins. Clock for NOR-flash.

Table 6.34 shows the internals of the QSPI Top.

Table 6.34: QSPI Top Signals

Signal	Width	Explanation
tbd	1	tbd

6.3.5 Service Requests of the QSPI

Interrupt sources are level-based and derived directly from kernel status or FIFO fill levels.

Table 6.35 shows the service requests of the QSPI.

Table 6.35: QSPI Service Requests (updated)

Bit	Service Request	IRQ	Explanation
0	DONE	DONE	Transfer completed (FSM reached DONE state)
1	RX_HALF	RX_HALF	RX FIFO $\geq$ half full
2	RX_EMPTY	RX_EMPTY	RX FIFO empty
3	TX_HALF	TX_HALF	TX FIFO $\leq$ half full (ready to be re-filled)
4	TX_EMPTY	TX_EMPTY	TX FIFO empty
5	ERROR	ERROR	Error latched (see STATUS register)
6	TIMEOUT	TIMEOUT	Timeout threshold reached
63:7	-	-	Reserved

In QSPI:

$$\text{MIS} = \text{RIS} \ \& \ \text{IMSC}$$

and then the external interrupt output is:

$$\text{IRQ} = | \text{MIS}$$

6.3.6 The QSPI Peripheral Kernel

Block Architecture -: The block architecture is like:

```

Wishbone
|
Register File + IRQ Logic (SRB)
|
QSPI Kernel FSM
|

```



```

TX FIFO / RX FIFO
|
SPI PHY (SCK / CS / IO[3:0])

```

The kernel is fully state-driven and does not depend on external events such as interrupts or bus timing.

Writing bit 0 of **QSPI0_CTRL.START** initiates a transaction if not busy.

Kernel State Machine -: The kernel operates in the following states:

- IDLE
- CMD_PHASE
- ADDR_PHASE
- DUMMY_PHASE
- DATA_PHASE
- DONE

State transitions occur only on the internal kernel clock.

Start Semantics: -: A transaction is started by:

$$start_s = (QSPI_CTRL.START == 1) \wedge (BUSY == 0)$$

The START bit is self-clearing.

START Write while BUSY -: Writing the START bit while BUSY is asserted is a no-operation (NOP): the write is silently discarded and no error is raised. This simplifies driver implementation (no mandatory pre-check required), but SW is responsible for polling BUSY or waiting for the DONE interrupt before initiating a new transaction if ordering is required.

The formal start condition remains:

$$start_s = (QSPI0_CTRL.START == 1) \wedge (QSPI0_STATUS.BUSY == 0)$$

Error Conditions -: Errors are latched in **QSPI0_STATUS.ERROR** and **QSPI0_RIS.ERROR**:

- Illegal command
- FIFO overflow/underflow
- XIP conflict
- Timeout

Errors persist until cleared via **QSPI0_ICR**.

Execute-In-Place (XIP) Mode -: When the XIP bit in **QSPI0_CTRL** is set, the QSPI controller automatically services read requests from the CPU bus without explicit SW intervention.

- XIP is only supported for read accesses. Write accesses from the bus while XIP is active are not forwarded to the flash and cause an ERROR.
- The opcode for XIP reads is taken from **QSPI0_CMD.OPCODE**. SW must configure this register with a suitable read command before enabling XIP (e.g. 0x6B for Quad Output Fast Read).
- The number of dummy cycles is taken from **QSPI0_DUMMY.DUMMY_CYC**.
- The ADDR_LEN, QUAD, DUAL, CPOL, CPHA, and CLKDIV settings apply as configured; these must be stable before XIP is enabled.
- While XIP is active, the START bit is ignored (writing START has no effect).
- XIP is terminated by clearing the XIP bit. The current flash transaction, if any, is completed before the controller returns to IDLE.
- An XIP conflict error is raised when SW attempts to write **QSPI0_CTRL.START** while XIP is active. This sets the ERROR bit in **QSPI0_STATUS** and in **QSPI0_RIS**.

Typical SW sequence to enable XIP:

1. Configure CLKDIV, CPOL, CPHA, QUAD/DUAL, ADDR_LEN as needed.
2. Write the desired read opcode to **QSPI0_CMD**.
3. Write the required dummy cycles to **QSPI0_DUMMY**.
4. Set XIP bit in **QSPI0_CTRL**. XIP becomes active immediately.

Design Rules -:

- The kernel FSM is fully state-driven.
- No FSM state depends on interrupt signals.
- Interrupts never control hardware behavior.
- All external behavior is driven by register state only.

Figure 6.4 shows the state machine of the QSPI. A simple Moore-FSM (Fig. 6.3): Additional to the FSM, a clock generator for `sck_gen` is required. This clock will be enabled or disabled by the FSM (clock enable cell). And more a few counters will be needed: a counter for the CMD-cycles (`cnt_cmd`), a counter for the ADDR-cycles (`cnt_adr`), a counter for the DUMMY-cycles (`cnt_dum`) and a counter for the DATA-cycles (`cnt_dat`). The counters end values will be either constants or loaded by a register. When writing to the TX-register a signal `tx` will be derived, this controls the direction of the data flow in the DATA state of the FSM.

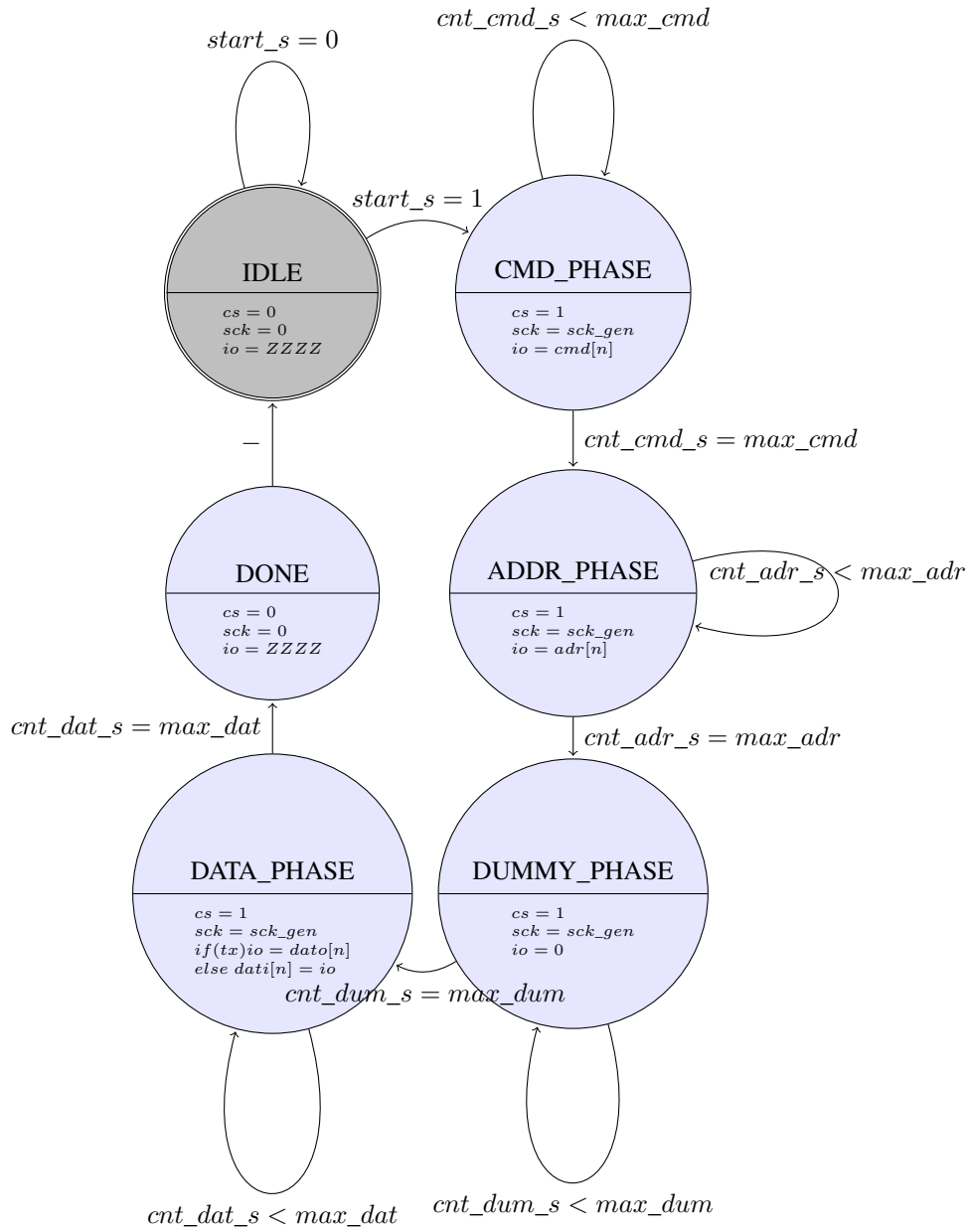


Figure 6.4: QSPI Kernel

6.3.7 Registers

In the following the registers of a QSPI peripheral are listed and described. Some registers' layout or availability depends on the generic feature set implemented for that QSPI's instantiation. Such registers and concerned bit fields are denoted by

the use of italic style.

QSPI Register Overview

Table 6.36: QSPI Register List

Register Group	Register Name	Register Symbol
System Registers	(Clock Control Register) Identification Register	(CLC) ID
Special Function Registers	(Run Control Register) QSPI Control register SPI command opcode QSPI target flash address QSPI transfer length in bytes QSPI dummy cycles QSPI status flags QSPI TX FIFO write QSPI RX FIFO read QSPI FIFO levels QSPI Clock divider (Baud Rate) QSPI Timeout	(RUN_CTRL) QSPI0_CTRL QSPI0_CMD QSPI0_ADDR QSPI0_LEN QSPI0_DUMMY QSPI0_STATUS QSPI0_TXDATA QSPI0_RXDATA QSPI0_FIFOSTAT QSPI0_CLKDIV QSPI0_TIMEOUT
Interrupt Registers	Interrupt Mask Control Register Raw Interrupt Status Register Interrupt Set Register Interrupt Clear Register Masked Interrupt Status Register	QSPI0_IMSC QSPI0_RIS QSPI0_ISR QSPI0_ICR QSPI0_MIS

Read-Write Features

- w: writable bit, by SW
- r: readable bit, by SW
- h: bit will be set by HW only

System Registers

QSPI_ID -: Identification register of this peripheral. A write will have no effect, a read will return the fixed ID. The ID will be defined on chip level.

ID register (in BPI)

[Reset value: 000100000000C'001_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
MOD_NUMBER															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
1	1	0	0	0	0	0	0	REV_NUMBER							
r	r	r	r	r	r	r	r	r							

Field	Bits	Type	Description
MOD_NUMBER	63:48	r	Module Identification Number.
-	47:16	r	Reserved
-	15:8	r	The value C0 indicates the peripheral as 64-bit module.
REV_NUMBER	7:0	r	Module Revision Number: Bits 7-0 are used for module revision numbering.

Special Function Registers

QSPI0_CTRL -: This register controls all transaction parameters of the QSPI interface:

- Start a transfer:

$$start_s = (QSPI0_CTRL.START == 1) \wedge (QSPI0_STATUS.BUSY == 0)$$

If START is written while BUSY is set, the write is ignored (no error). The START bit is self-clearing; a read always returns 0.

- TX and RX FIFOs can be flushed independently via TX_FLUSH and RX_FLUSH. These bits are self-clearing. Flushing is only allowed when BUSY = 0; otherwise the flush is ignored.
- Single (DUAL=0, QUAD=0), Dual (DUAL=1, QUAD=0) or Quad (QUAD=1) SPI data mode.
- 3-Byte or 4-Byte flash address selectable via ADDR_LEN.

- Chip select can be kept asserted between transactions (CS_HOLD).

[Reset value: 000000000000000002_H]

[illegible]

Field	Bits	Type	Description
-	63:14	r	Reserved
RX_FLUSH (RF)	13	w	Flush RX FIFO. Self-clearing. Only effective when BUSY=0.
TX_FLUSH (TF)	12	w	Flush TX FIFO. Self-clearing. Only effective when BUSY=0.
ADDR_LEN (AL)	11	rw	Address length. 0 = 3-Byte (24 bit), 1 = 4-Byte (32 bit).
CPOL (CP)	10	rw	Clock polarity. 0 = idle low (Mode 0/2), 1 = idle high (Mode 1/3).
CPHA (CH)	9	rw	Clock phase. 0 = sample on first edge (Mode 0/3), 1 = sample on second edge (Mode 1/2). Note: only combinations CPOL=CPHA=0 and CPOL=CPHA=1 are supported by the reference flash devices.
DUAL (DU)	8	rw	Enable dual-wire data mode (1-1-2). Effective only when QUAD=0.
CS_HOLD (CS)	7	rw	Keep CS asserted between consecutive transactions.
XIP (XI)	6	rw	Enable execute-in-place mode. See Section 6.3.6 for details.
DDR (DD)	5	rw	Enable double data rate.
QUAD (QU)	4	rw	Enable quad-wire data mode (1-1-4 or 1-4-4). Overrides DUAL.
START (ST)	3	w	Start transfer. Writing 1 initiates a transaction if BUSY=0. Self-clearing.
-	2:0	r	Reserved.

QSPI0_CMD -: The (Q)SPI opcode will be stored.

Command register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	OPCODE							
r	r	r	r	r	r	r	r	rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
OPCODE	7:0	rw	(Q)SPI opcode

QSPI0_ADDR -: Address register, 4-byte. For target address in a NOR-flash.

Address register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADDR															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
ADDR															
rw															

Field	Bits	Type	Description
-	63:32	r	Reserved
ADDR	31:0	rw	Target address in NOR flash

QSPI0_LEN -: Transfer length in bytes.

LEN register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LEN															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
LEN															
rw															

Field	Bits	Type	Description
-	63:32	r	Reserved
LEN	31:0	rw	Transfer length in bytes.

QSPI0_DUMMY -: Register stores the amount of dummy cycles.

DUMMY register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	DUMMY_CYC					
r	r	r	r	r	r	r	r	r	r	rw					

Field	Bits	Type	Description
-	63:6	r	Reserved
DUMMY_CYC	5:0	rw	Amount of dummy cycles

QSPI0_STATUS -: Status register of the QSPI peripheral.

STATUS register (in BPI)

[Reset value: 000000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	TI	ER	EM	FU	DO	BU
r	r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:6	r	Reserved
TI	5	rh	TIMEOUT, timeout occurred
ER	4	rh	ERROR, error latched
EM	3	rh	FIFO_EMPTY, RX-FIFO empty
FU	2	rh	FIFO_FULL, TX-FIFO full
DO	1	rh	DONE, Transfer completed
BU	0	rh	BUSY, Kernel active

QSPI0_TXDATA -: TX-FIFO write register.

TX register (in BPI)

[Reset value: 000000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
TX FIFO															
rw															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
TX FIFO															
rw															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TX FIFO															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
TX FIFO															
rw															

Field	Bits	Type	Description
TX	63:0	rw	TX FIFO write

QSPI0_RXDATA -: RX-FIFO read register.

RX register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
RX FIFO															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
RX FIFO															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RX FIFO															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
RX FIFO															
r															

Field	Bits	Type	Description
RX FIFO	63:0	r	RX FIFO read

QSPI0_FIFOSTAT -: Displays the status of the FIFO.

FIFO status register (in BPI)

[Reset value: 000000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
RX_LEV								TX_LEV							
rh								rh							

Field	Bits	Type	Description
-	63:16	r	Reserved
RX_LEV	15:8	rh	RX FIFO level
TX_LEV	7:0	rh	TX FIFO level

QSPI0_CLKDIV -: The Clock Divider Register controls the frequency of the generated SCK clock. The SCK frequency is derived from the kernel clock as:

$$f_{SCK} = \frac{f_{kernel}}{2 \times (CLKDIV + 1)}$$

With $f_{kernel} = 125$ MHz this yields the following example values:

Table 6.37: CLKDIV Examples ($f_{kernel} = 125$ MHz)

CLKDIV	f_{SCK}	Comment
0x00	62.5 MHz	Maximum SCK (check flash device spec)
0x01	31.25 MHz	
0x03	15.625 MHz	Suitable for slow read without dummy cycles
0xFF	244.1 kHz	Minimum SCK

The register must not be changed while a transfer is active (BUSY=1).

Clock Divider register (in BPI) [Reset value: 0000000000000003_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0								CLKDIV							
r								rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
CLKDIV	7:0	rw	Clock divider value. Reset value 0x03 gives $f_{SCK} \approx 15.6$ MHz as a safe default for most NOR-Flash devices without dummy cycles.

QSPI0_TIMEOUT -: The Timeout Register defines the maximum number of kernel clock cycles that the FSM may remain in any non-IDLE state before a TIMEOUT error is raised. If the counter reaches zero while the kernel is active (BUSY=1), the FSM is forced back to IDLE, the CS line is deasserted, and the TIMEOUT bit is set in the STATUS and RIS registers.

Setting TIMEOUT_THR to 0x00000000 disables the timeout mechanism.

A typical value for a 125 MHz kernel clock and a maximum allowed transaction time of 1 ms is: TIMEOUT_THR = 125 000 = 0x0001E848.

Timeout register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIMEOUT_THR[31:16]															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
TIMEOUT_THR[15:0]															
rw															

Field	Bits	Type	Description
-	63:32	r	Reserved
TIMEOUT_THR	31:0	rw	Timeout threshold in kernel clock cycles. 0x00000000 = timeout disabled.

Interrupt Registers

Interrupt sources are level-based and derived directly from kernel status.

Flow of Interrupt Requests -: Figure 6.5 shows the flow of a single interrupt request, which comes from the Peripheral Kernel or from the FIFO Controller.

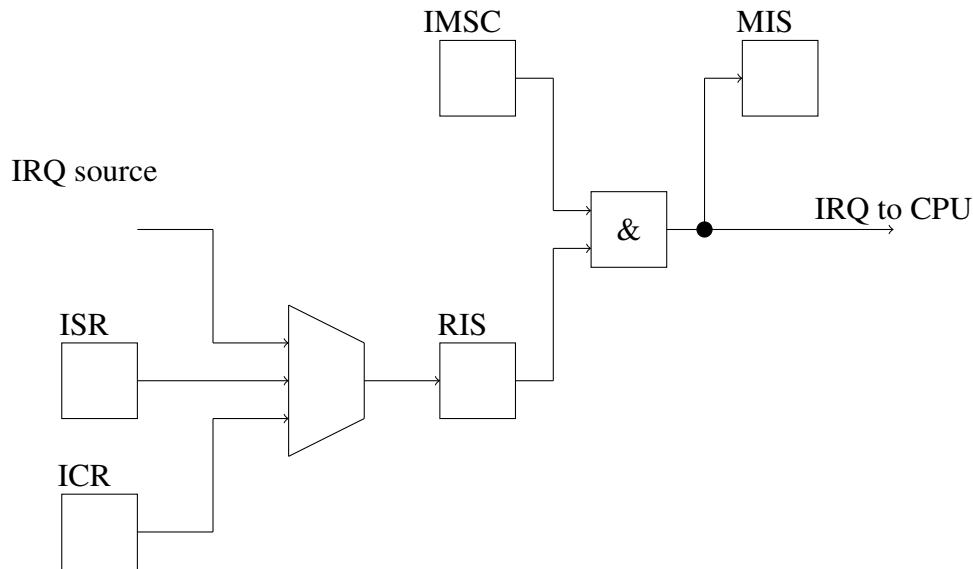


Figure 6.5: Interrupt Request Flow

A pulse on an interrupt request `<xyz>_IRQ` sets the corresponding interrupt status bit in the Raw Interrupt Status Register RIS. The interrupt status bits can be set with the Interrupt Set Register ISR or can be cleared with the Interrupt Clear Register ICR.

The Interrupt Mask Control Register IMSC enables or disables the level sensitive interrupts to the ICU. The Masked Interrupt Status Register MIS gives the current masked status value of the corresponding interrupt.

If an IMSC bit is disabled while its interrupt is active, then the interrupt for the ICU will be removed, but only until this IMSC bit is enabled again, unless the interrupt has been cleared in the mean time.

If the IMSC bit is not enabled when an interrupt source becomes active, then the interrupt bit will only be set in the RIS register. If the corresponding IMSC bit is later enabled, the interrupt will consequently become active in the MIS register.

Before enabling an interrupt in the MIS bit, it is good practice to always first clear the corresponding interrupt in the RIS via ICR.

In QSPI:

$$MIS = RIS \& IMSC$$

and then the external interrupt output is:

$$IRQ = |MIS$$

QSPI0_RIS -: The Raw Interrupt Status Register (RIS) is read-only. It reflects the unmasked current interrupt status. Writing has no effect.

- 0 – No Interrupt pending
- 1 – Interrupt pending

Raw Interrupt Status Register (in BPI) [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	TI	ER	TE	TH	EM	HA	DO
r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:7	r	Reserved
TI	6	rh	TIMEOUT, Timeout threshold reached
ER	5	rh	ERROR, Error latched
TE	4	rh	TX_EMPTY, TX FIFO empty
TH	3	rh	TX_HALF, TX FIFO $\leq$ half full
EM	2	rh	RX_EMPTY, RX FIFO empty
HA	1	rh	RX_HALF, RX FIFO $\geq$ half full
DO	0	rh	DONE, Transfer completed

QSPI0_IMSC -: The Interrupt Mask Control Register (IMSC) enables or disables individual interrupt sources. Write 1 to unmask (enable); write 0 to mask (disable).

- 0 – Interrupt disabled (masked)
- 1 – Interrupt enabled (unmasked)

Interrupt Mask Control Register (in BPI) [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	TI	ER	TE	TH	EM	HA	DO
r	r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw	rw	rw

Field	Bits	Type	Description
-	63:7	r	Reserved
TI	6	rw	TIMEOUT
ER	5	rw	ERROR
TE	4	rw	TX_EMPTY
TH	3	rw	TX_HALF
EM	2	rw	RX_EMPTY
HA	1	rw	RX_HALF
DO	0	rw	DONE

QSPI0_MIS -: The Masked Interrupt Status Register (MIS) is read-only. It returns the logical AND of RIS and IMSC, i.e. the effective interrupt status visible to the CPU.

- 0 – No Interrupt pending
- 1 – Interrupt pending

Masked Interrupt Status Register (in BPI) [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	TI	ER	TE	TH	EM	HA	DO
r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:7	r	Reserved
TI	6	rh	TIMEOUT
ER	5	rh	ERROR
TE	4	rh	TX_EMPTY
TH	3	rh	TX_HALF
EM	2	rh	RX_EMPTY
HA	1	rh	RX_HALF
DO	0	rh	DONE

QSPI0_ICR -: The Interrupt Clear Register (ICR) is write-only. Writing 1 to a bit clears the corresponding interrupt in both RIS and MIS. Writing 0 has no effect. Reading returns 0.

Note: If a data transfer is handled via interrupt requests (this chip has no DMA), the interrupt should be cleared via ICR only after the FIFO has been drained or refilled, to avoid losing the interrupt if the condition is still active.

- 0 – No change
- 1 – Clear Interrupt

Interrupt Clear Register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	TI	ER	TE	TH	EM	HA	DO
r	r	r	r	r	r	r	r	r	w	w	w	w	w	w	w

Field	Bits	Type	Description
-	63:7	r	Reserved
TI	6	w	TIMEOUT
ER	5	w	ERROR
TE	4	w	TX_EMPTY
TH	3	w	TX_HALF
EM	2	w	RX_EMPTY
HA	1	w	RX_HALF
DO	0	w	DONE

QSPI0_ISR -: The Interrupt Set Register (ISR) is write-only and is provided for software testing of the interrupt path. Writing 1 forces the corresponding RIS bit to 1 (and consequently MIS if IMSC is set). Writing 0 has no effect. Reading returns 0.

- 0 – No change
- 1 – Force Interrupt (test only)

Interrupt Set Register (in BPI)

[Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	TI	ER	TE	TH	EM	HA	DO
r	r	r	r	r	r	r	r	r	w	w	w	w	w	w	w

Field	Bits	Type	Description
-	63:7	r	Reserved
TI	6	w	TIMEOUT
ER	5	w	ERROR
TE	4	w	TX_EMPTY
TH	3	w	TX_HALF
EM	2	w	RX_EMPTY
HA	1	w	RX_HALF
DO	0	w	DONE

Chapter 7

Test and Debug

Not yet

Chapter 8

Electrical Characteristics

Not yet

Chapter 9

Memory Map and Registers

Not yet

Bibliography

- [1] The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2, <https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf>, Accessed: 2024-02-23, 2017.
- [2] RISC-V Guide, <https://marks.page/riscv/>, Accessed: 2024-02-23, 2024.
- [3] Five EmbedDev, <https://five-embeddev.com/riscv-isa-manual/latest/instr-table.html>, Accessed: 2024-02-23, 2024.
- [4] G. Lehmann, B. Wunder, and M. Selz, Schaltungsdesign mit VHDL, Synthese, Simulation und Dokumentation: Franzis, 1994, ISBN: 978-3772361630. [Online]. Available: https://www.itiv.kit.edu/downloads/Buch_gesamt.pdf.